Ejercicios unidad 2. PostGIS

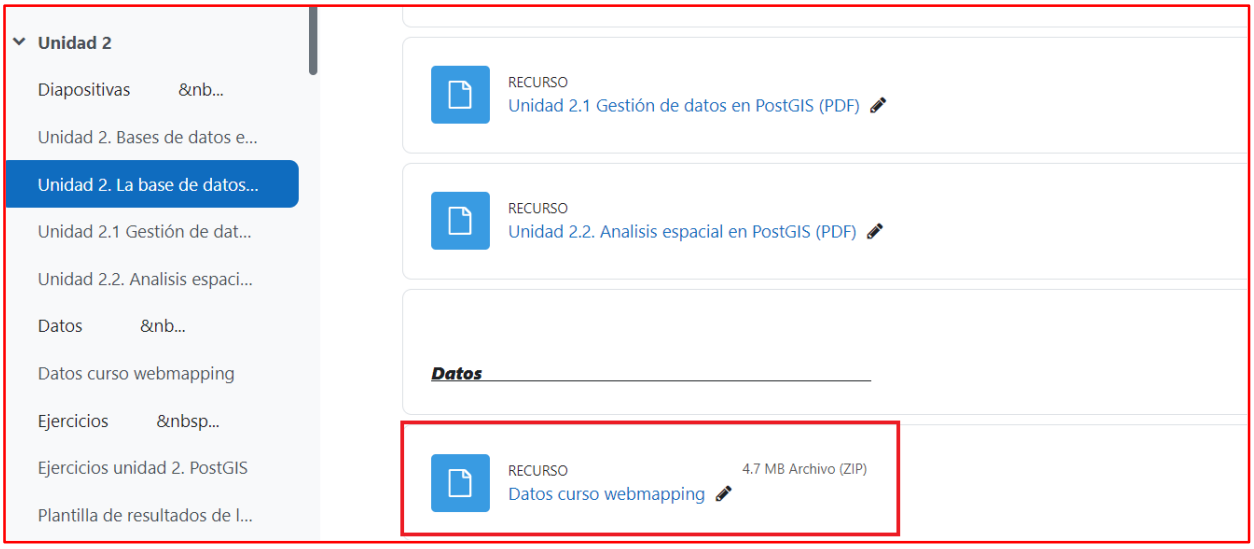


CONTENIDO

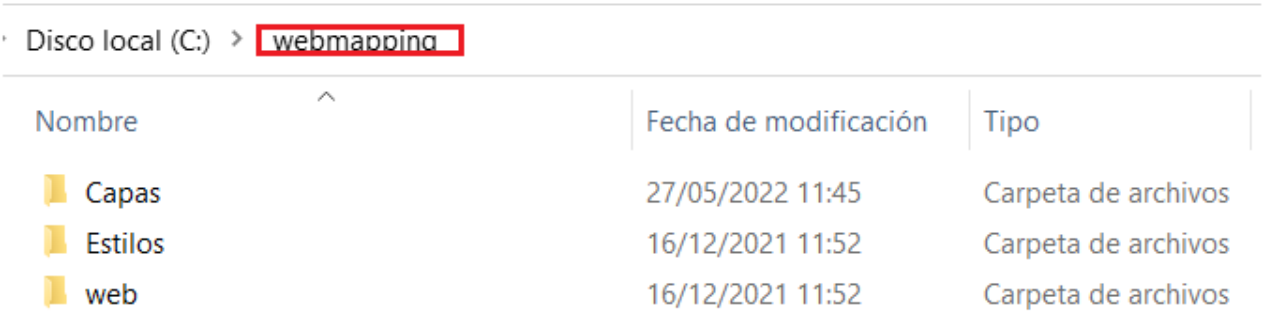
Descarga de datos 3
Descarga e instalación de PostgreSQL y PostGIS 4
Instalación de PostgreSQL 4
Instalación de PostGIS 12
Creación de una base de datos espacial PostGIS 21
Iniciar pgAdmin y conexión con PostGIS 21
Crear una base de datos 24
Creación de una base espacial desde la línea de comandos 28
¿Dónde guarda PostGIS las bases de datos? 28
El esquema de tablas y vistas de PostGIS 29
Tabla spatial_ref_sys 29
Vistas geometry_columns y geography_columns 30
La importación y exportación de datos en PostGIS 32
La herramienta *PostGIS 3 Shapefile and DBF Loader Exporter* 32
Conexión desde un SIG a la base de datos PostGIS 39
Importar cartografía a PostGIS con el administrador de BBDD de QGIS 43
Exportar tablas de PostGIS a archivos vectoriales con el administrador de BBDD de QGIS 48
Visualización de los datos espaciales en PostGIS 51
El visor de geometrías integrado en pgAdmin 51
Las consultas espaciales y funciones básicas 55
Cómo acceder a las funciones espaciales 55
Creación de una tabla habilitada para operaciones espaciales 56
Consultas sencillas SQL 60
Cómo seleccionar datos de una tabla 61
Uso de alias 62
Funciones espaciales 65

Descarga de datos

Todos los datos y archivos necesarios para este curso se encuentran en el archivo ***webmapping.zip*** que puedes descargar de la plataforma de formación.



Descomprime la carpeta en un directorio **que no contenga espacios**, para evitar problemas. Lo ideal es en C:\webmapping\



Ejercicio nº1. Haz una captura de pantalla como la de arriba con los datos extraídos en un directorio sin espacios.

Los datos vectoriales que vamos a utilizar en el curso provienen de Natural Earth, a escala 1:110m. Se trata de las capas de:

- Países: *ne_110m_admin_0_countries.shp*
- Lugares poblados: *ne_110m_populated_places.shp*
- Ríos*: ne_110m_rivers_lake.gpkd*

La capa *hotdogs.gpkg* es una capa en formato GeoPackage de elaboración propia.

La capa ráster proviene de Natural Earth, a escala 1:50m y contiene el relieve coloreado basándose en la cubierta terrestre: *relieve.tiff*

Durante el curso haremos referencia a este directorio llamándolo `<webmapping>`.

Información del paquete de datos:

- `<webmapping>\Capas`: contiene archivos .shp, .gpkg y tif.
- `<webmapping>\Estilos:` contiene los archivos SLD que aplicaremos a algunas de nuestras capas.
- `<webmapping>\web`: contiene un archivo HTML con dos visores web.

Descarga e instalación de PostgreSQL y PostGIS

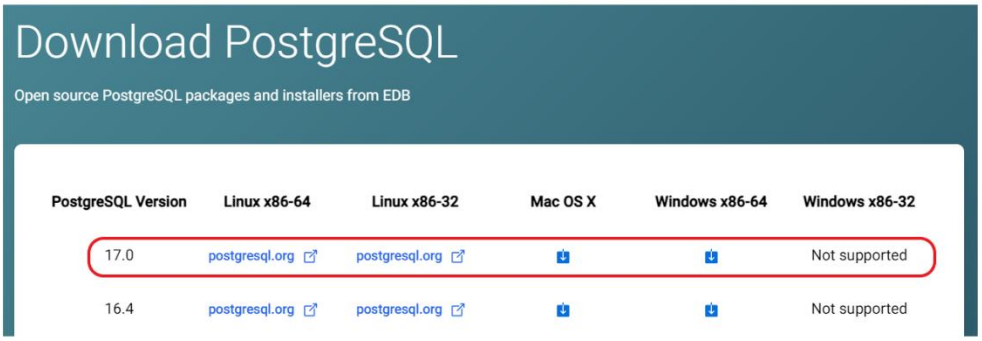
Instalación de PostgreSQL

En primer lugar, si tienes previamente una instalación con la suite de *OpenGeo* procede a desinstalarla.

Accedemos a la página web de descargas de PostgreSQL:

https://www.enterprisedb.com/downloads/postgres-postgresql-downloads

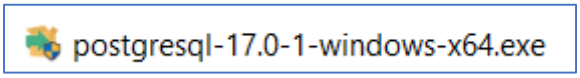
- Selecciona la versión de tu sistema operativo:



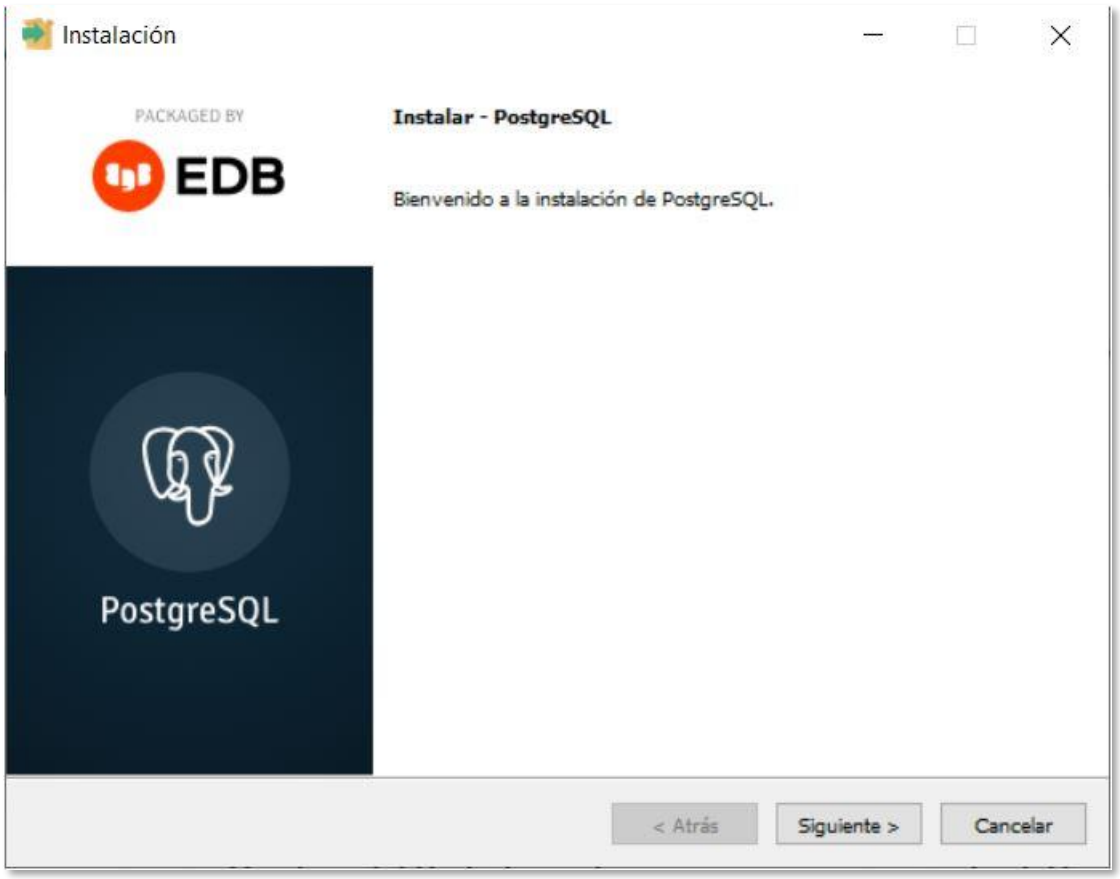
Clic en el botón de descarga según tu sistema operativo.

La descarga tardará unos minutos.

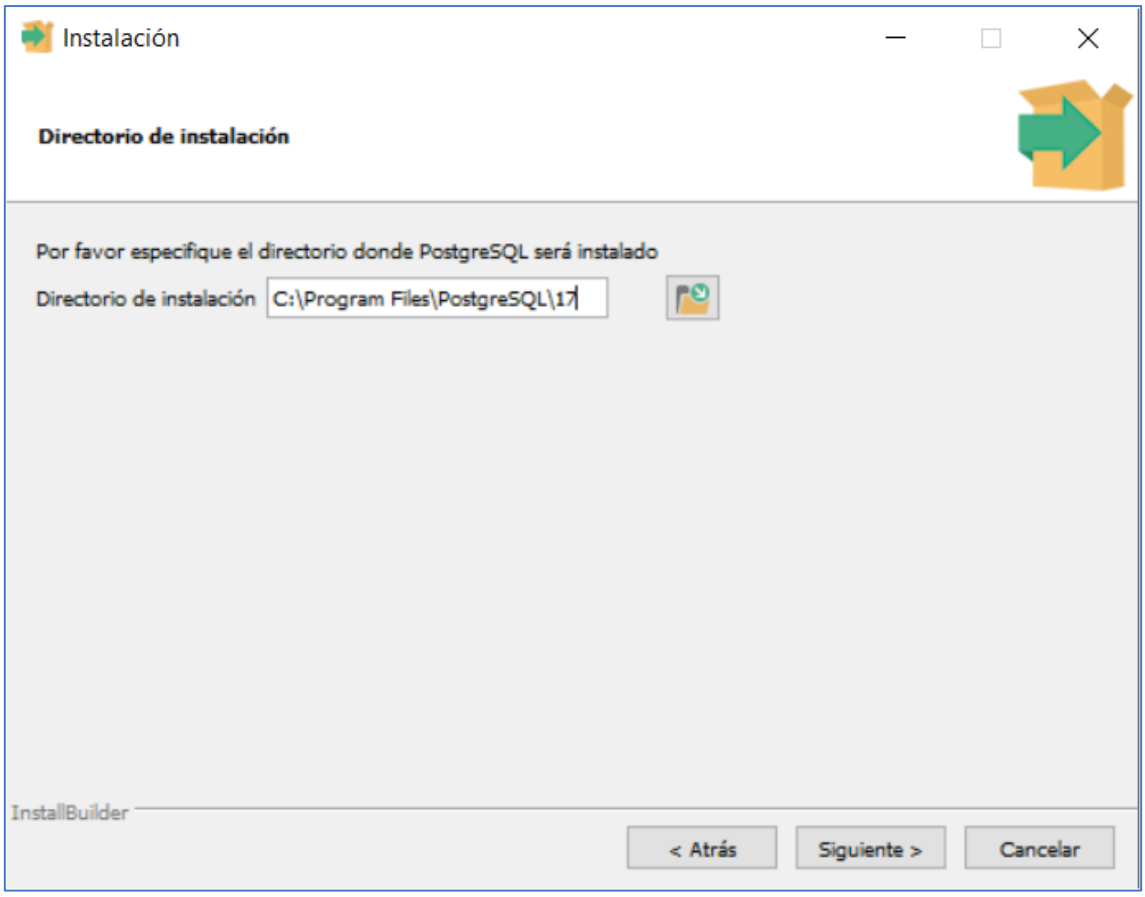
Una vez descargado ejecútalo haciendo doble clic sobre el archivo.



Nos aparecerá en pantalla la ventana de bienvenida a la instalación de *PostgreSQL*. Haz clic en *Siguiente.*

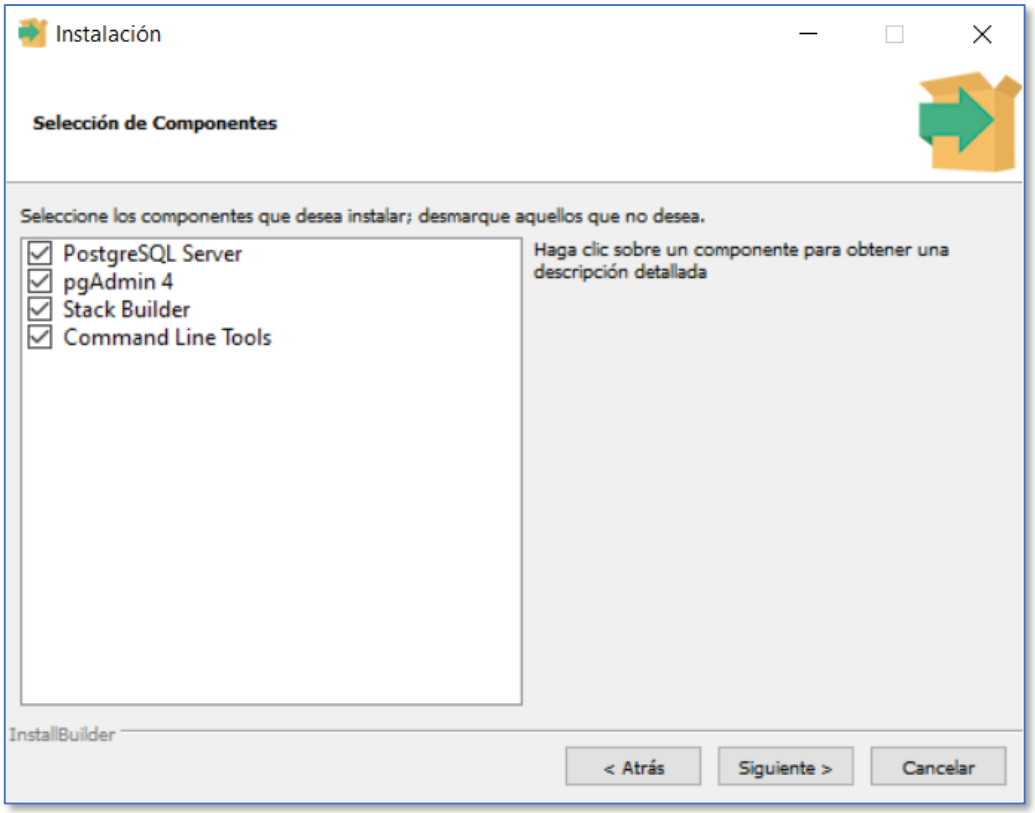


En la siguiente ventana se nos solicita el directorio de instalación. Nuestra recomendación es que instales *PostgreSQL* **directamente en tu disco duro C:/**



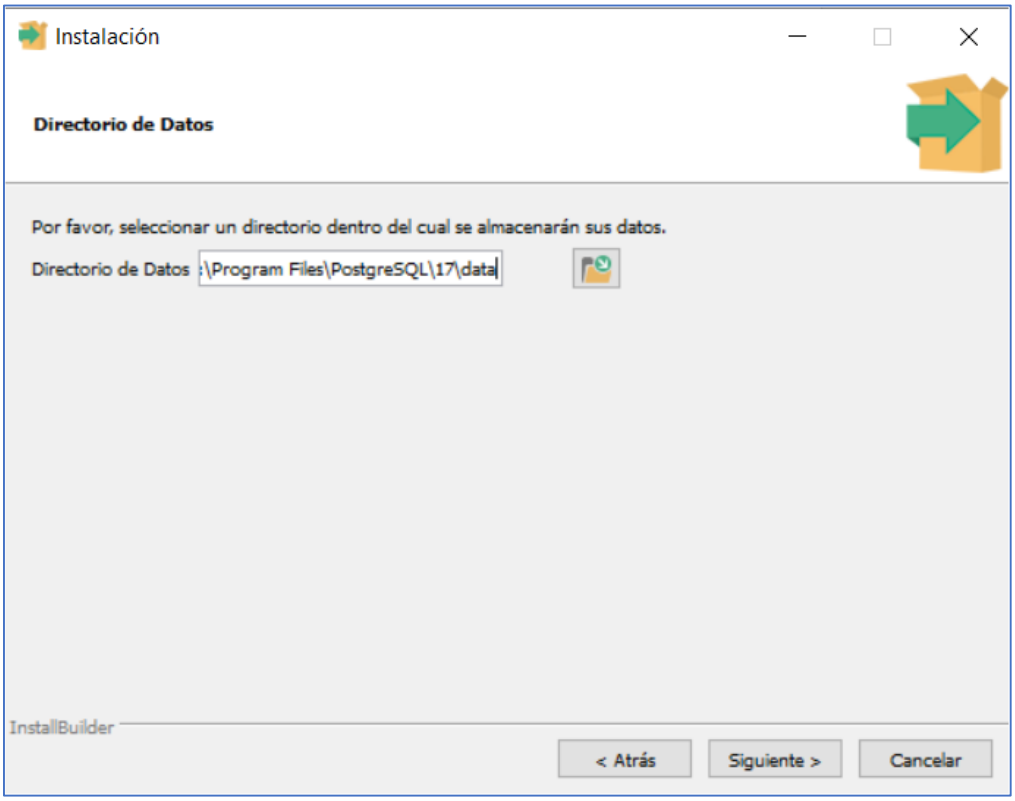
Haz clic en **Siguiente**.

En la siguiente ventana dejaremos marcadas todas las casillas:



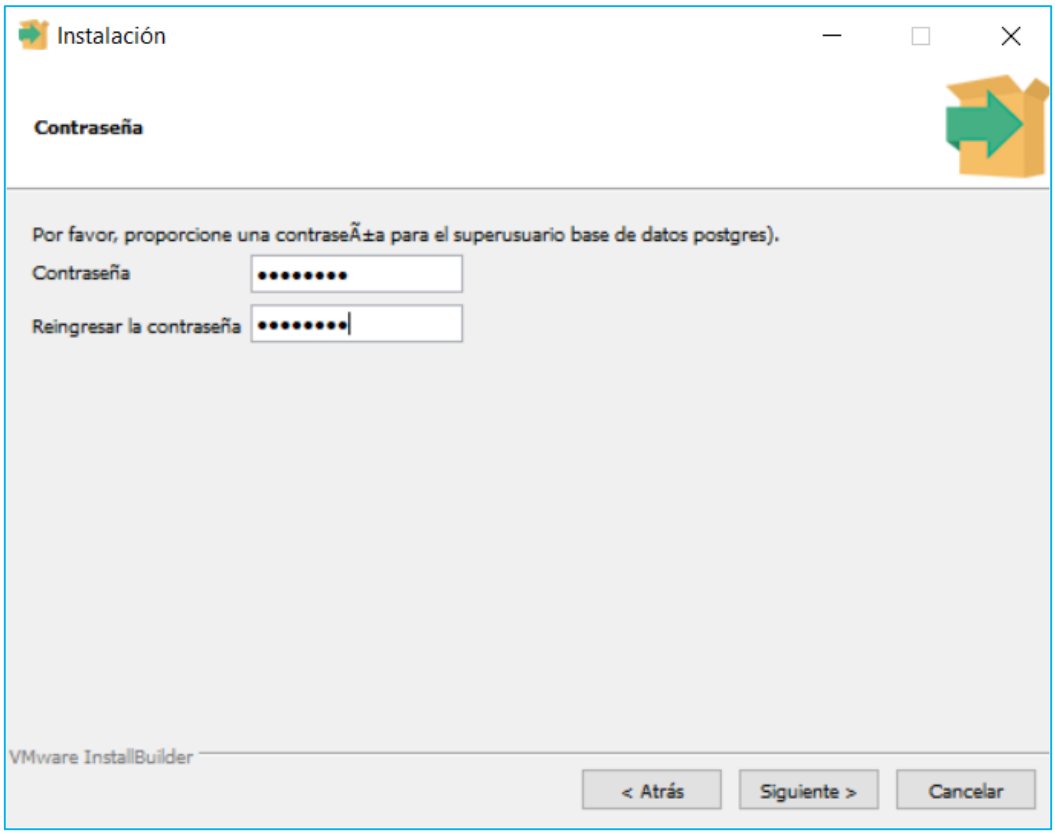
Haz clic en *Siguiente.*

La siguiente ventana nos pide el directorio en el que se guardarán los datos, **guárdalos en el mismo directorio que escogiste en el paso anterior**.

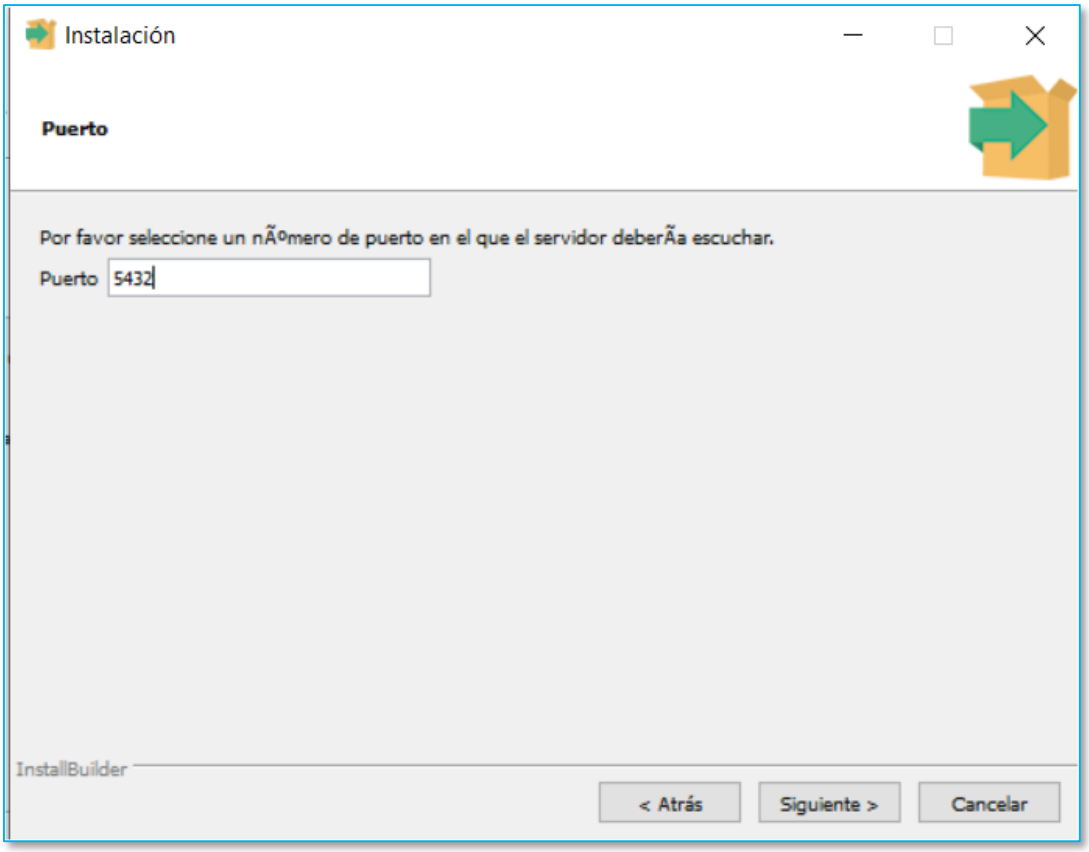


Haz clic en *Siguiente.*

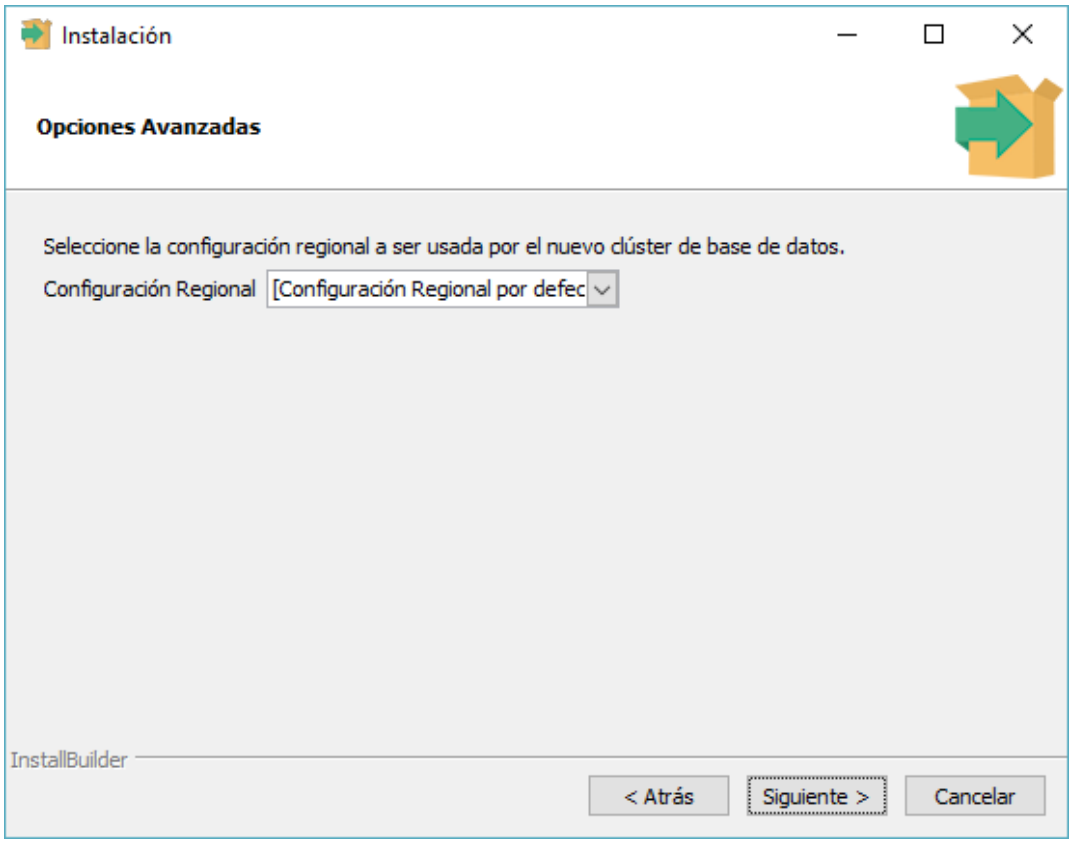
A continuación, el asistente de instalación nos requiere una contraseña. Te recomendamos que escojas **postgres**, haz clic en *Siguiente.*



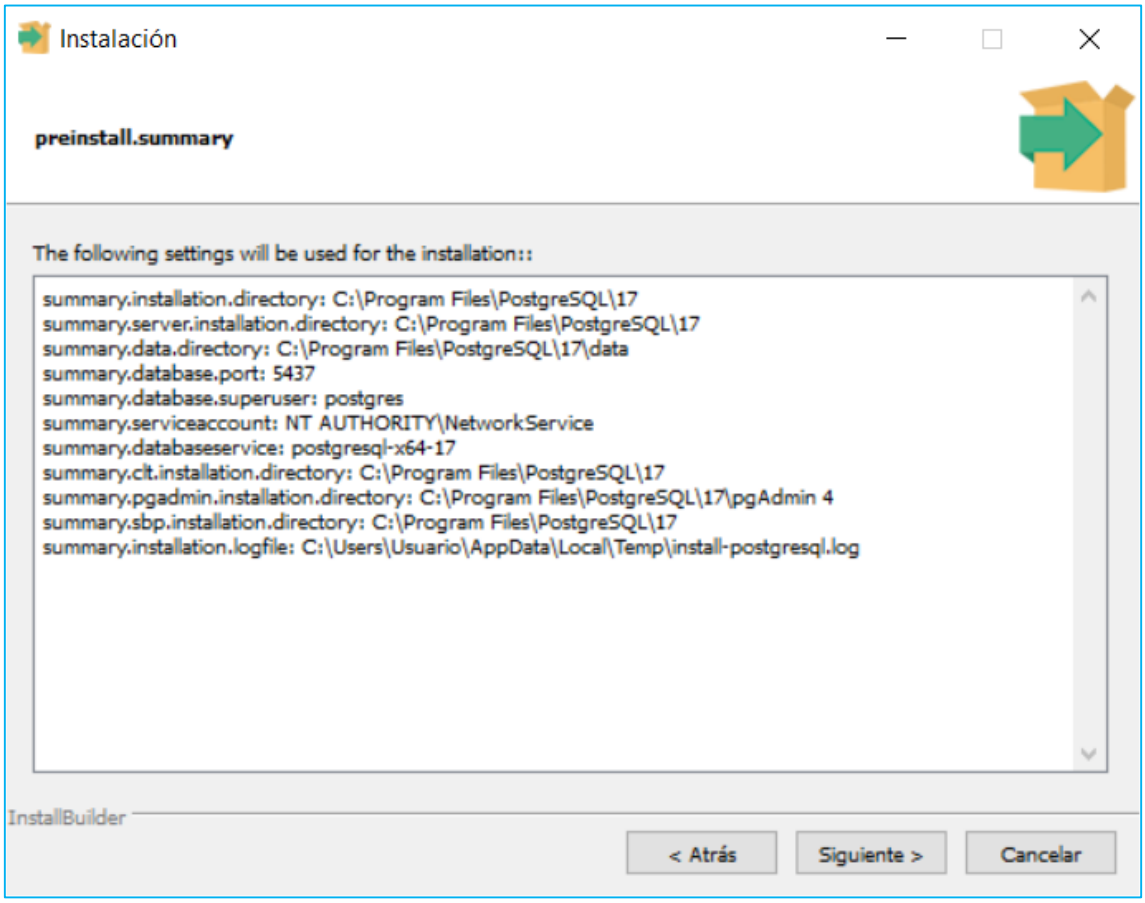
Selecciona el puerto **5432** por el que debería escuchar peticiones y haz clic en *Siguiente.*



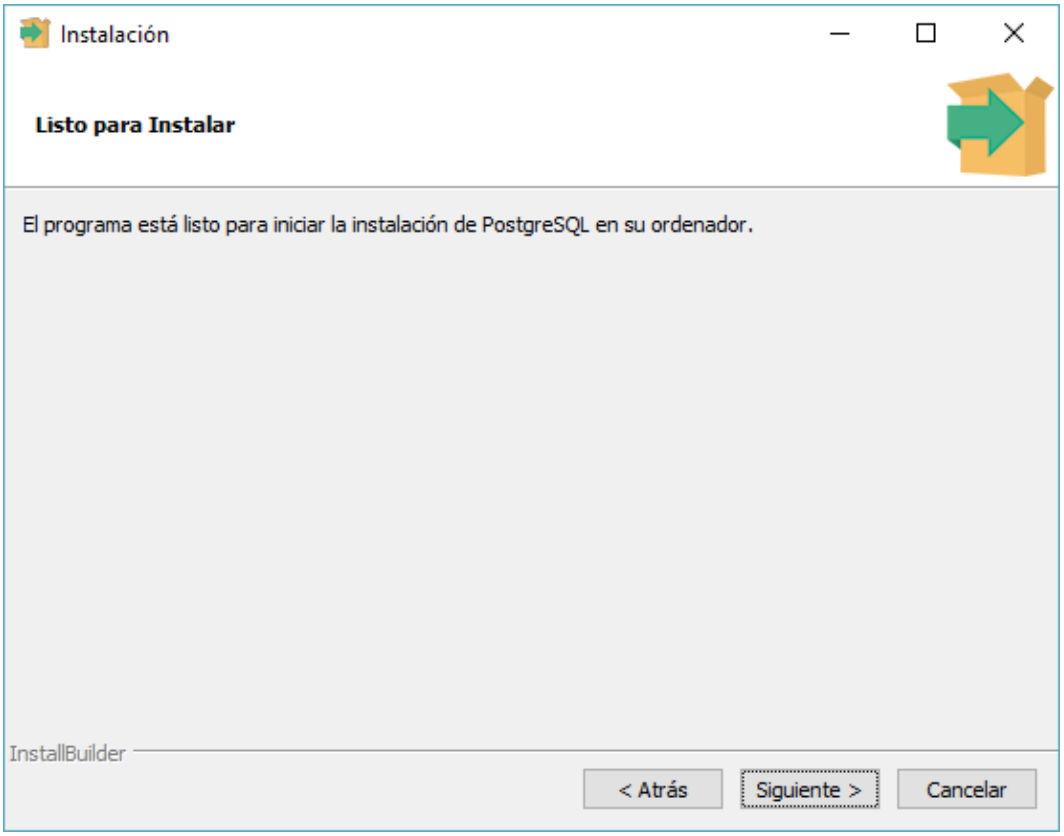
En Opciones Avanzadas escoge la *Configuración Regional por defecto* y haz clic en *Siguiente.*



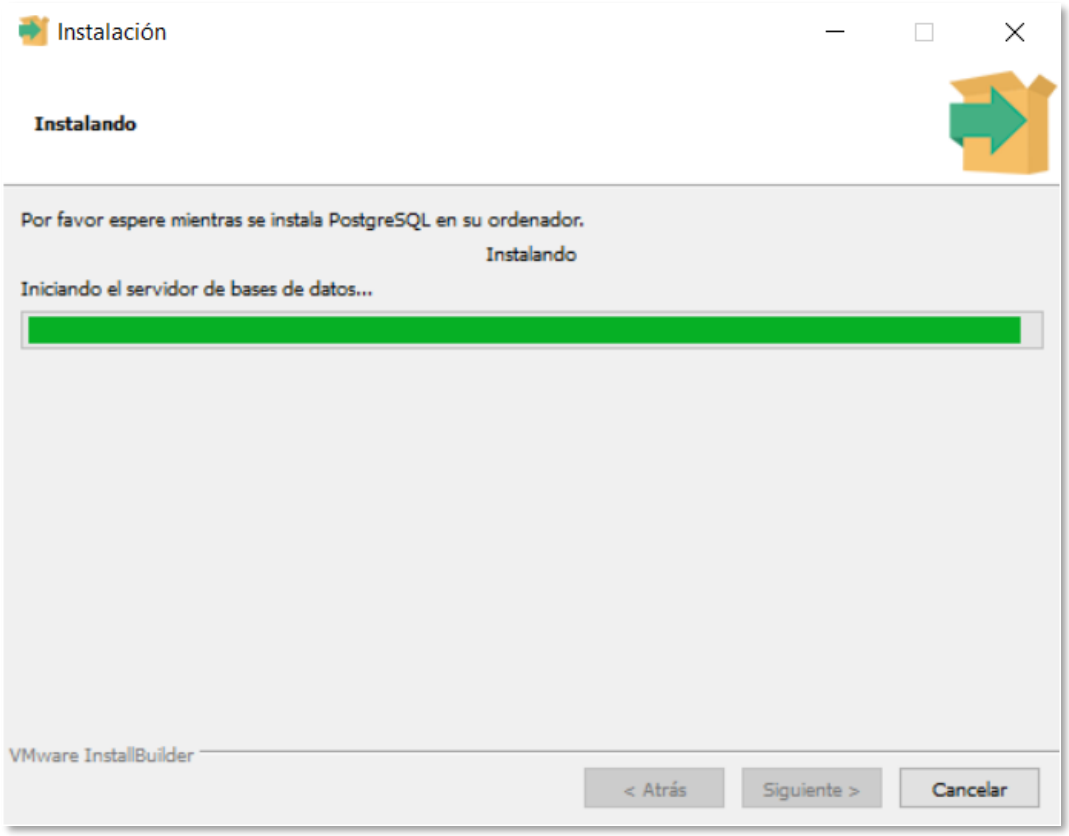
El asistente ya tiene la información necesaria para proceder con la instalación, nos aparecerá un resumen:



Haz clic en *Siguiente* para que comience el proceso.

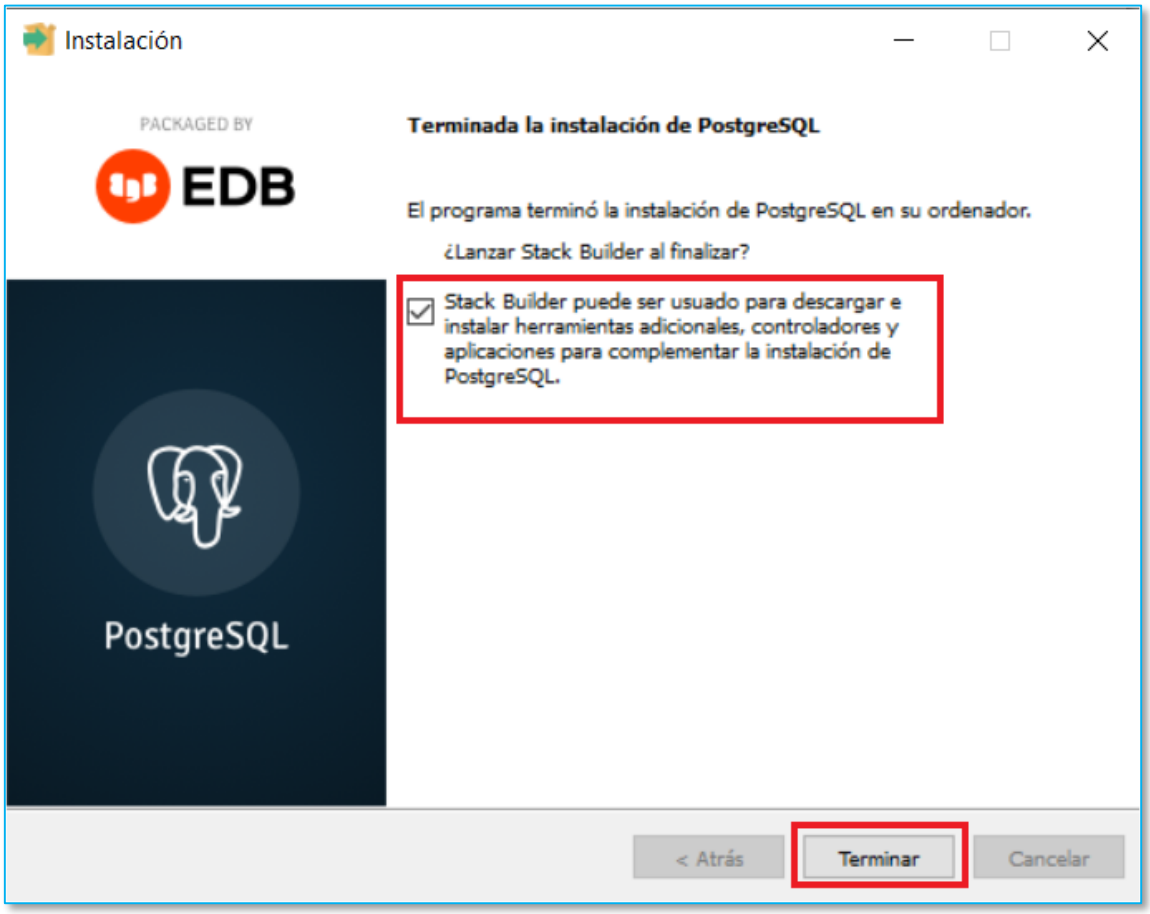


Una nueva ventana nos indicará la evolución del proceso.

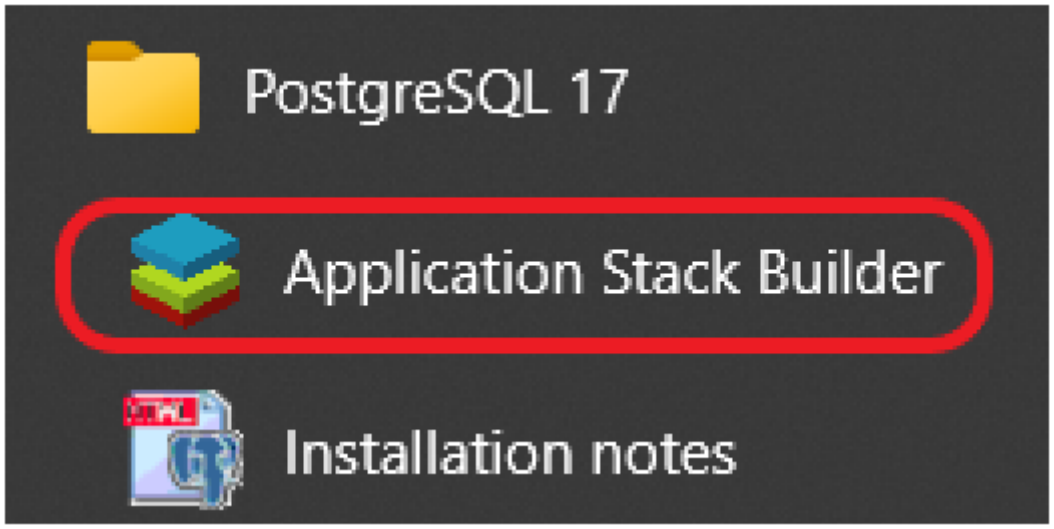


Una vez finalice la instalación marca la casilla de verificación para lanzar la aplicación ***Stack Builder*** y haz clic en ***Terminar***.

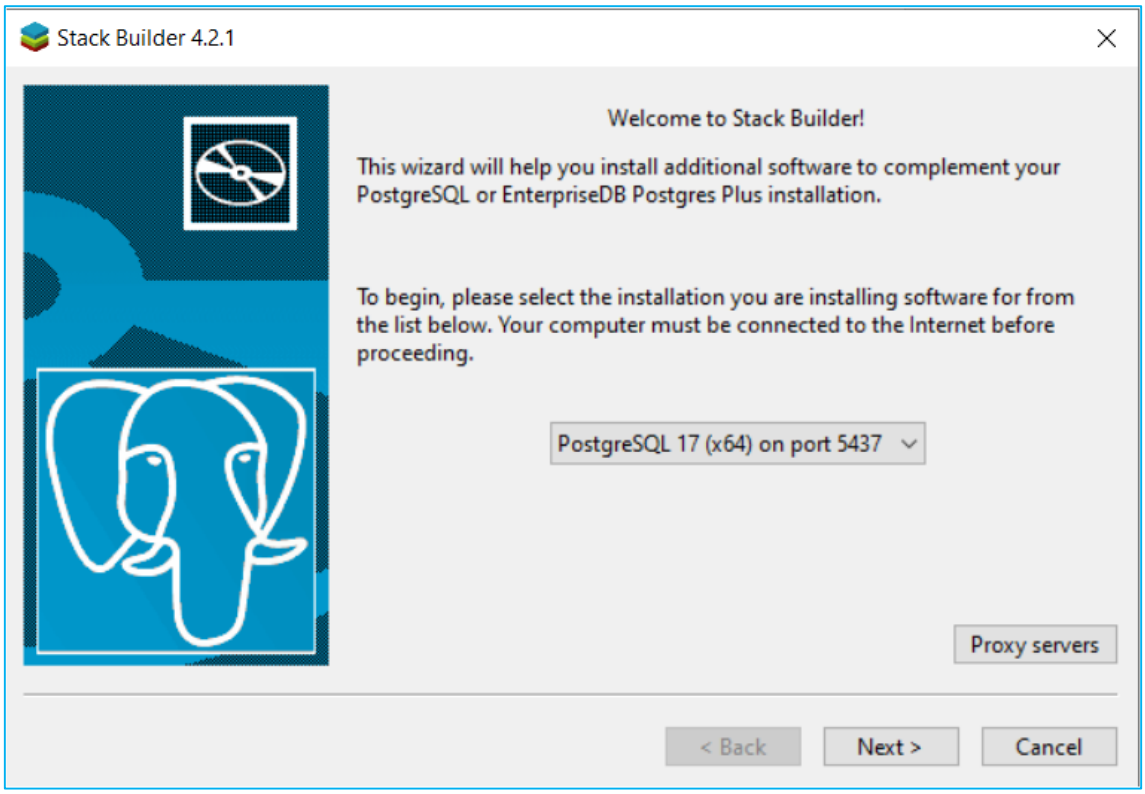
Instalación de PostGIS



Si no aparece dicha casilla, accede al menú de Inicio y arranca la aplicación ***Stack Builder***...



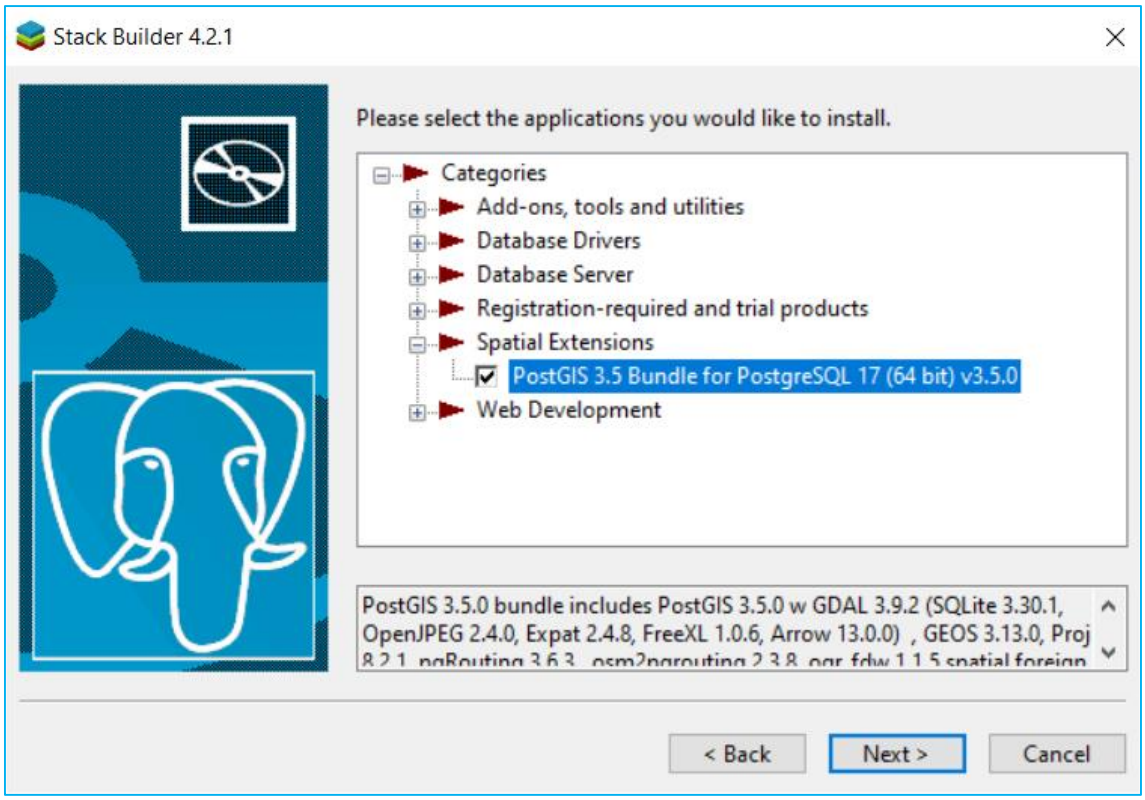
Una vez haya arrancado *Stack Builder* indica en el desplegable **PostgreSQL 17 (x64) on port 5432.**



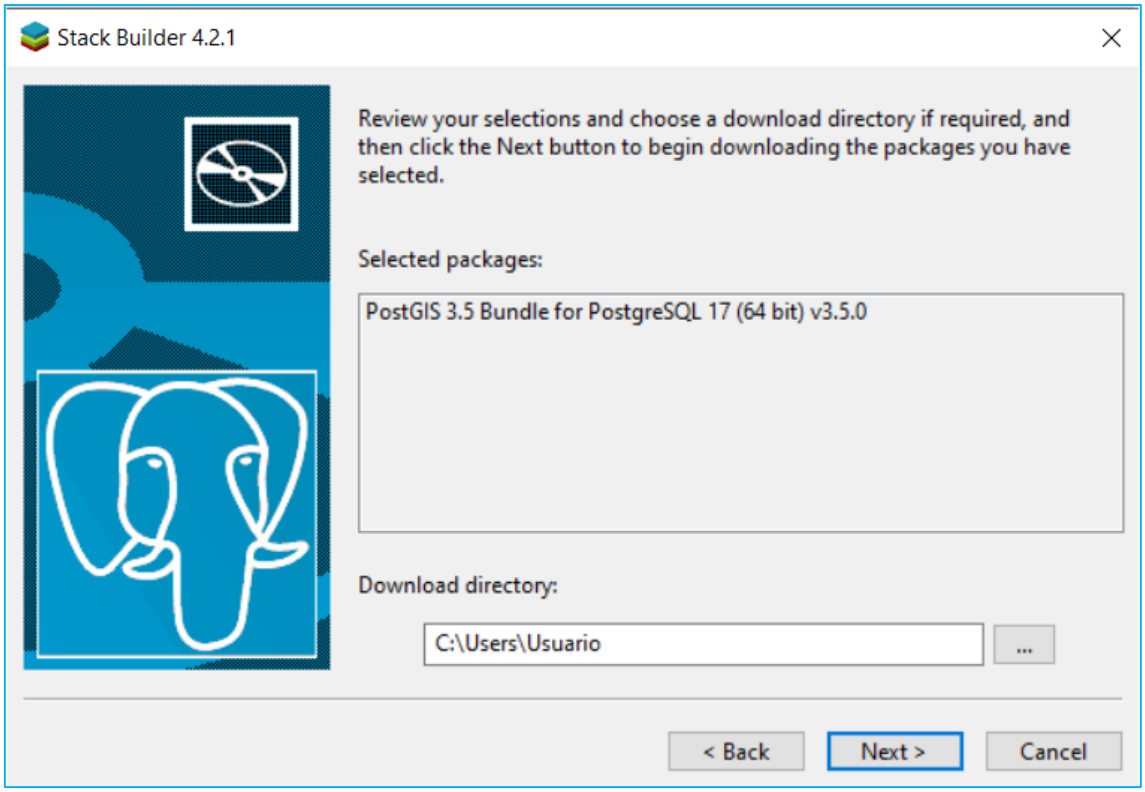
Haz clic en ***Next***.

Despliega ***Spatial Extensions*** y selecciona la versión *PostGIS 3.5***.**

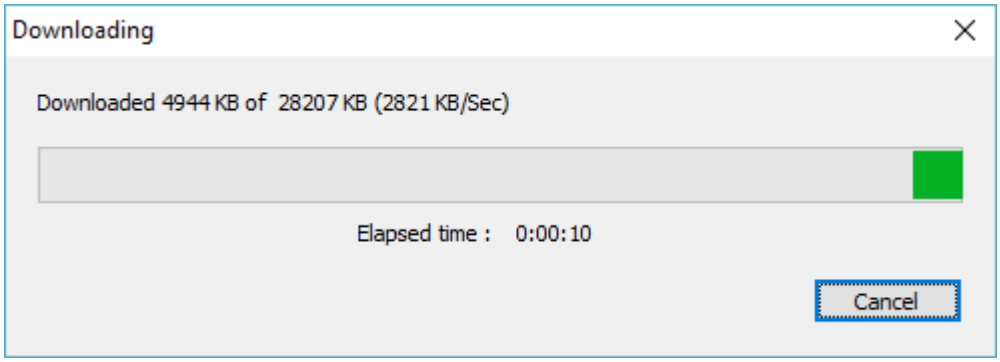
Haz clic en ***Next***.

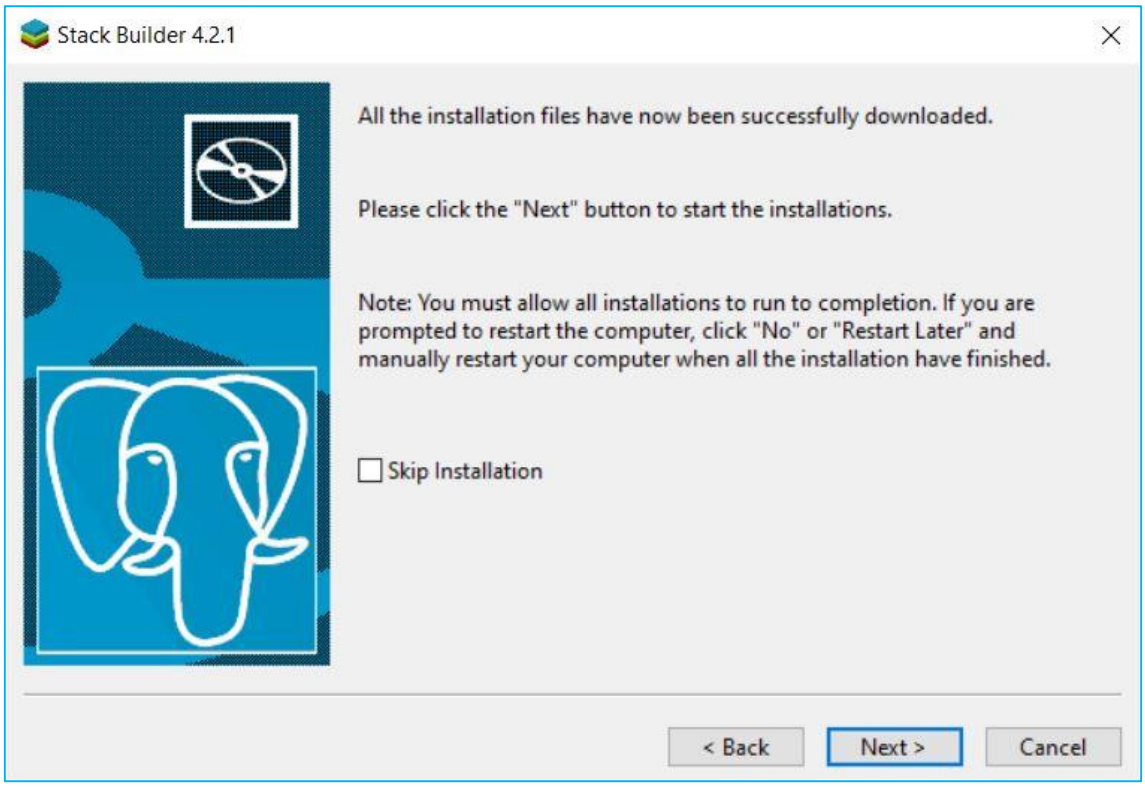


En la siguiente ventana el asistente solicita un directorio de instalación, deja el que aparece por defecto y haz clic en *Next.*



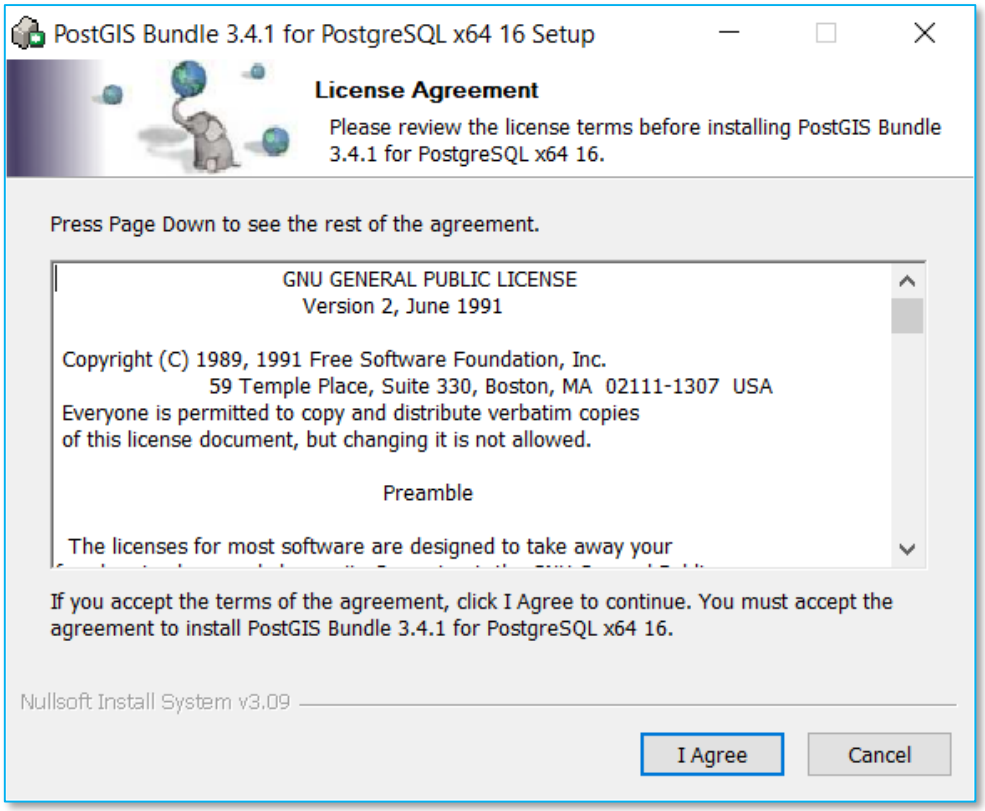
Comenzará el proceso de descarga.



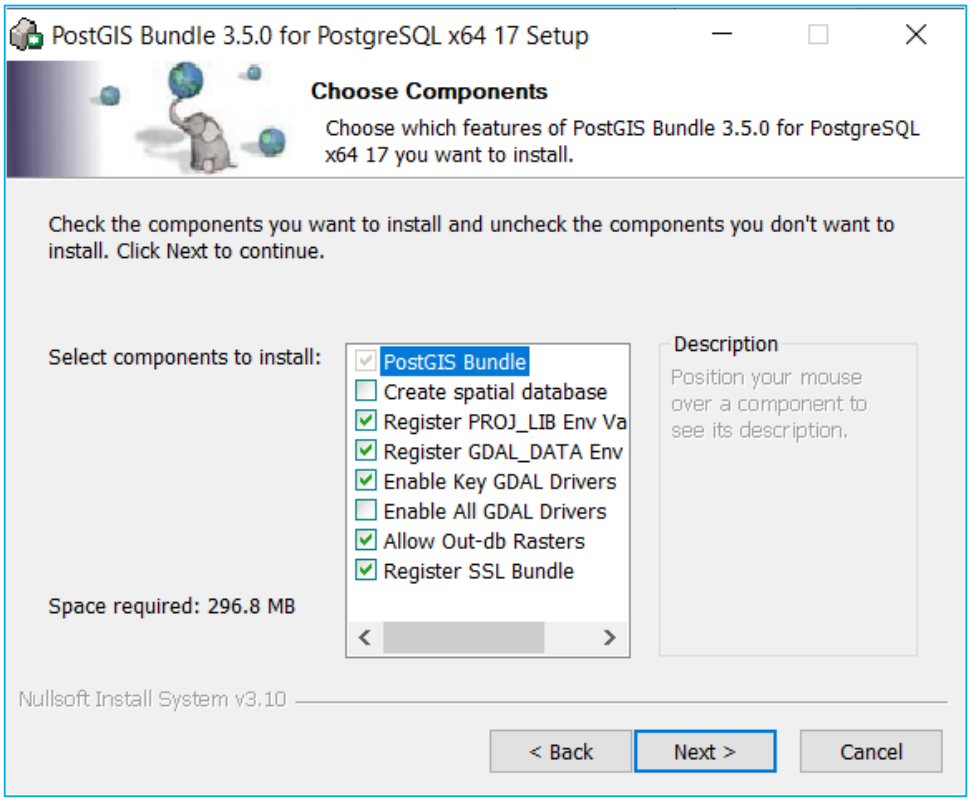


Clic en **Next**.

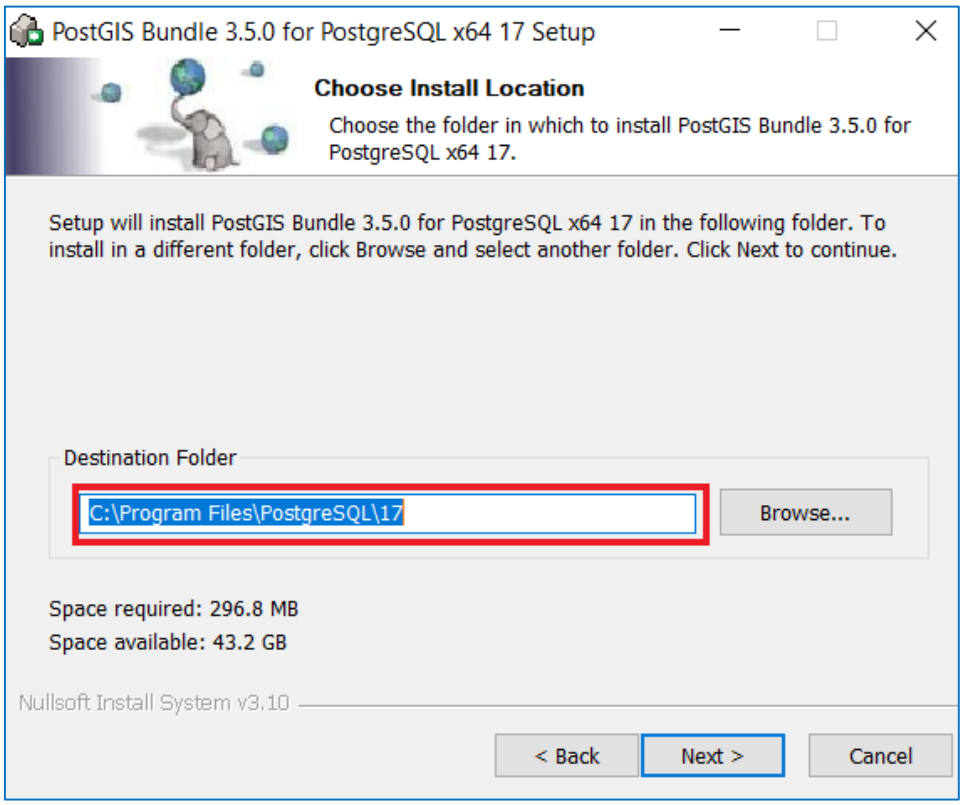
Acepta el acuerdo de licencia como requisito para que este proceso se inicie.



En la siguiente ventana aparecerá marcado el componente *PostGIS,* junto con otras opciones, haz clic en *Next.*



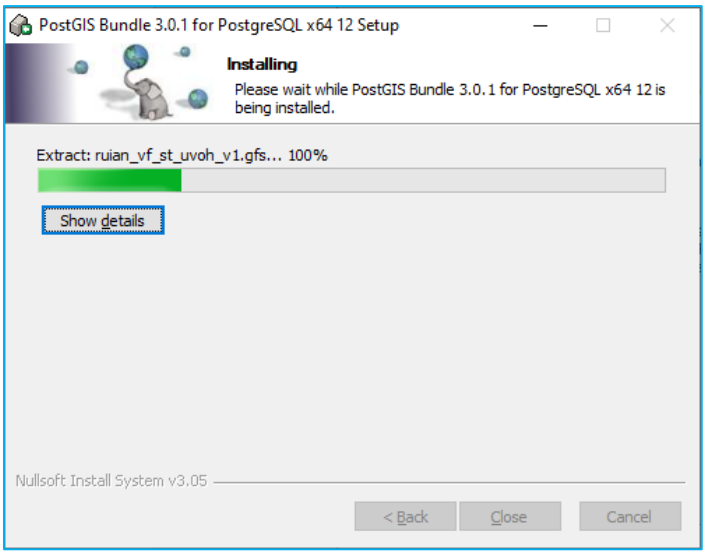
Confirma que el directorio de destino de la instalación es en el que se encuentra la instalación de *PostgreSQL* previamente realizada (quizá debas editar e introducir la ruta C:\PostgreSQL\17).



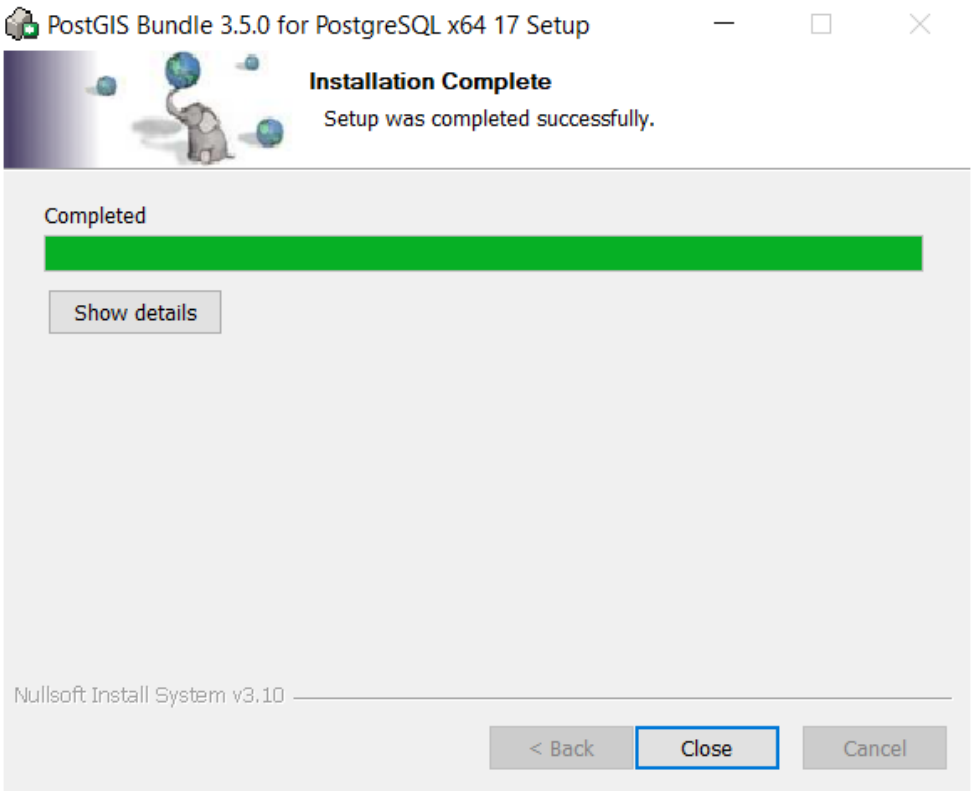
Haz clic en *Next* para comenzar el proceso de instalación.

IMPORTANTE: Si has creado la instalación de *PostgreSQL* directamente sobre tu disco duro C:/, asegúrate de instalar *PostGIS* en el mismo directorio, de lo contrario te encontrarás con errores durante la formación.

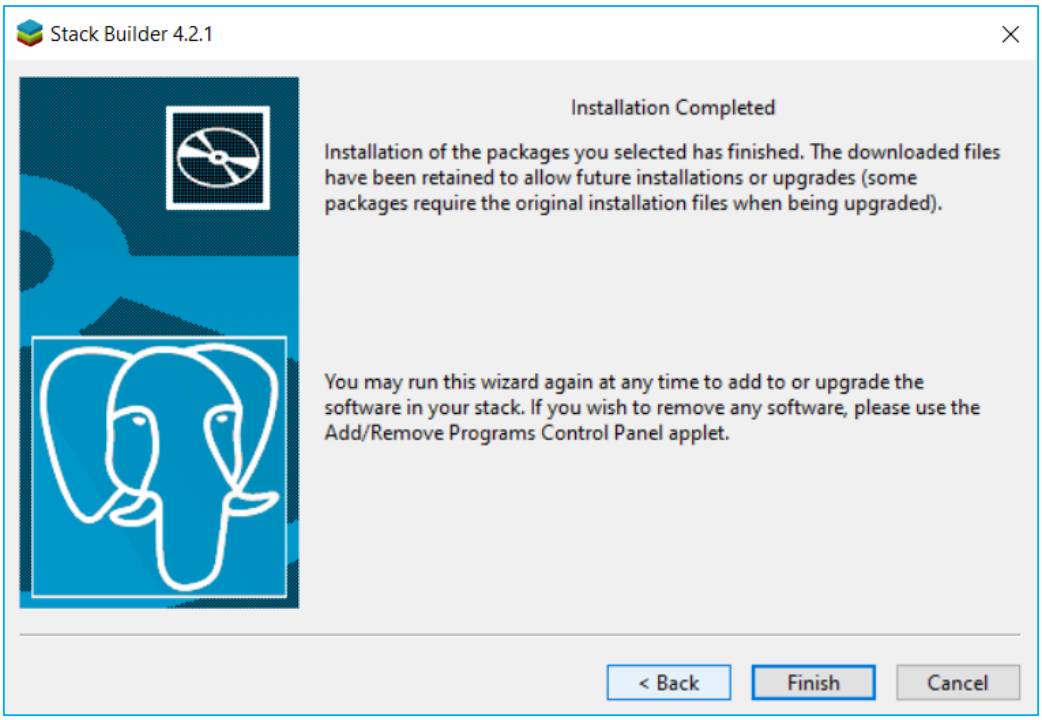
Se iniciará el proceso de instalación de PostGIS.



Una vez hecho esto, quedará finalizada la instalación, haz clic en *Close* en la ventana de Instalación de PostGIS.

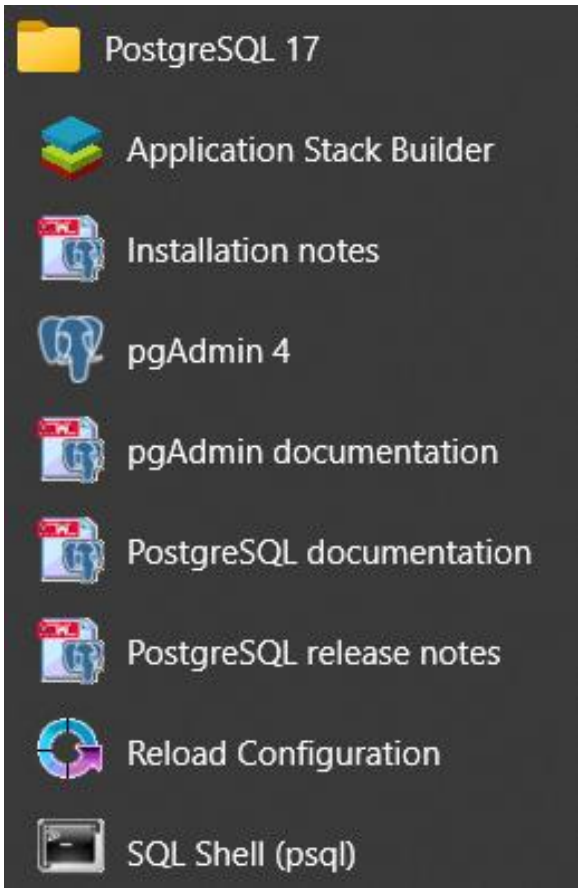


Cierra el asistente *Stack Builder* haciendo clic en *Finish.*


Stack Builder 4.2.1
Installation Completed
Installation of the packages you selected has finished. The downloaded files have been retained to allow future installations or upgrades (some packages require the original installation files when being upgraded).
You may run this wizard again at any time to add to or upgrade the software in your stack. If you wish to remove any software, please use the Add/Remove Programs Control Panel applet.
< Back
Finish
Cancel

Listo!

Una vez instalado abriremos la aplicación utilizando el nuevo acceso directo:



Ejercicio nº2. Desde el menú Inicio ve a programas > **pgAdmin 4** y abre la aplicación pgAdmin 4. Haz una captura de pantalla de pgAdmin 4 abierto en el navegador.

Creación de una base de datos espacial PostGIS

En esta sección, con algunos conceptos de bases de datos espaciales en nuestro haber, vamos a utilizar el cliente GUI (*graphical user interface*) pgAdmin para conectar y crear una base de datos con capacidades espaciales.

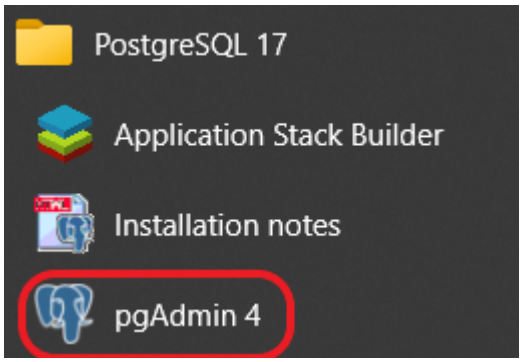
PostgreSQL tiene varios componentes de administración *front-end*, esto quiere decir que interactúa con el usuario de manera amigable.

- psql es una herramienta de línea de comandos para entrar en consultas SQL.
- pgAdmin, es una herramienta gratuita y de código abierto PostgreSQL que se ha hecho muy popular debido a su *front-end* gráfico.

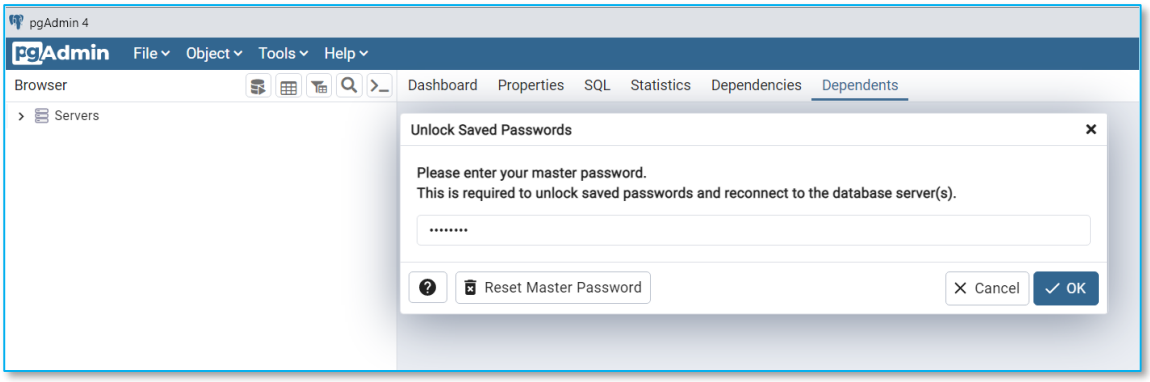
En este curso vamos a usar mayoritariamente pgAdmin, pero todas las consultas que se pueden hacer en pgAdmin también se pueden hacer en línea de comandos con psql.

Iniciar pgAdmin y conexión con PostGIS

1. En Windows pgAdmin se puede iniciar desde el menú Inicio > PostgreSQL 17 > pgAdmin 4.



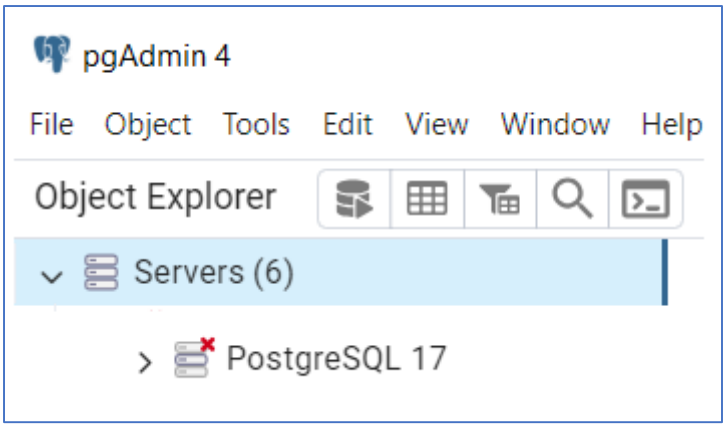
   Se abre una vista en la que nos pide la contraseña.



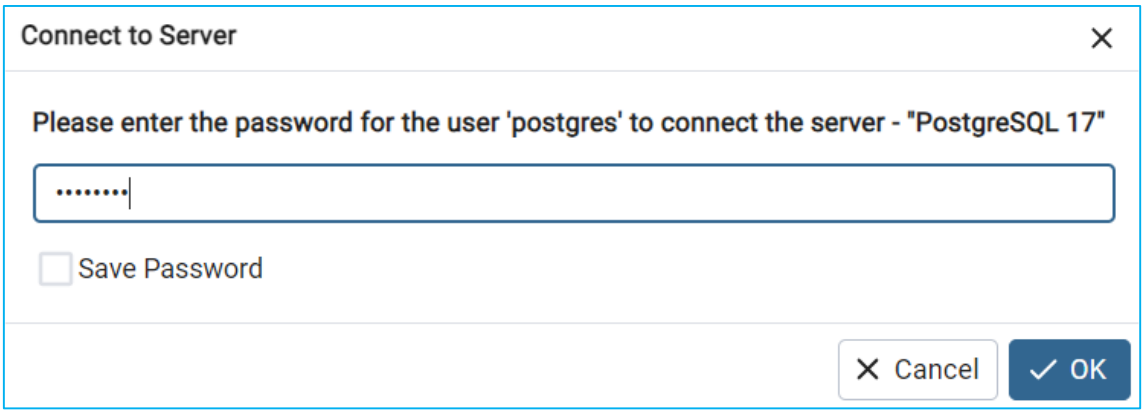
Escribe: **postgres**.

2. Al iniciar, deberías tener un servidor PostgreSQL preconfigurado en pgAdmin con el nombre **PostgreSQL 17**. Haz clic sobre Browser.

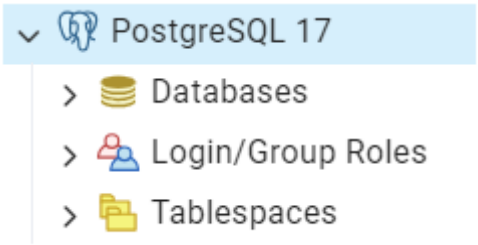
3. Haz **clic** en la entrada de postgreSQL17.



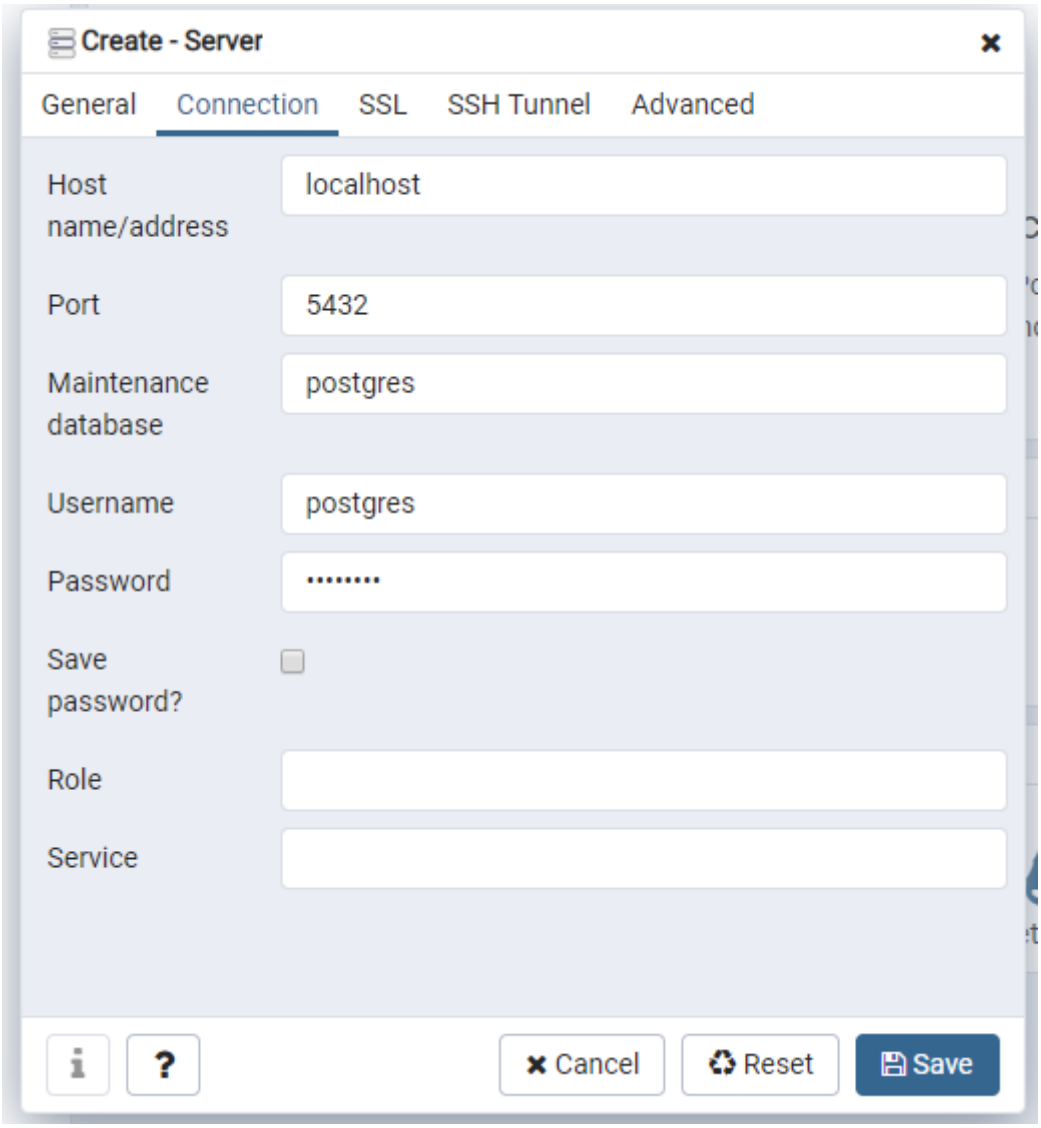
4. Introduce la contraseña **postgres** y marca la casilla de guardar el password (*Save Password*).



Se desplegará el árbol:



Nota
Si no tienes una entrada para PostgreSQL necesitarás crear una nueva conexión. Ve a Servers > Create > Server..., y registra un nuevo servidor en **localhost** y el puerto **5432.**

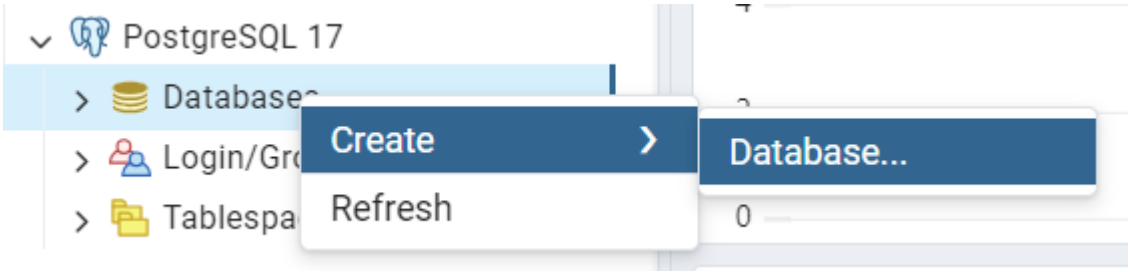


Parámetros de conexión en Windows / OS X

Crear una base de datos

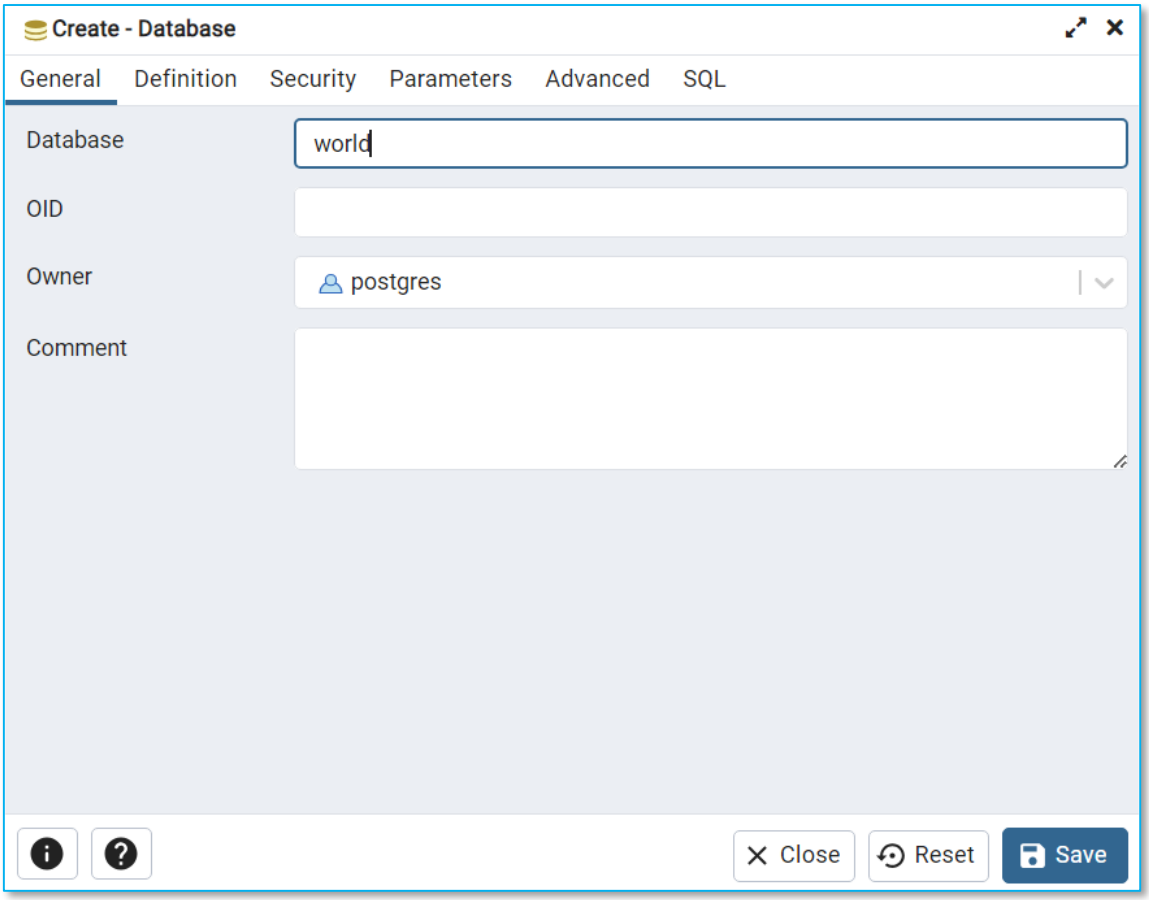
En este apartado vamos a realizar el proceso para crear una base de datos y dotarla de capacidades espaciales.

Haz clic derecho sobre el elemento `Databases` y selecciona `Create > Database...`



Rellena el formulario creación de una nueva base de datos como se muestra a continuación.

- **Name:** `world`
- **Owner:** `postgres`



Haz clic en el botón ***Save***.

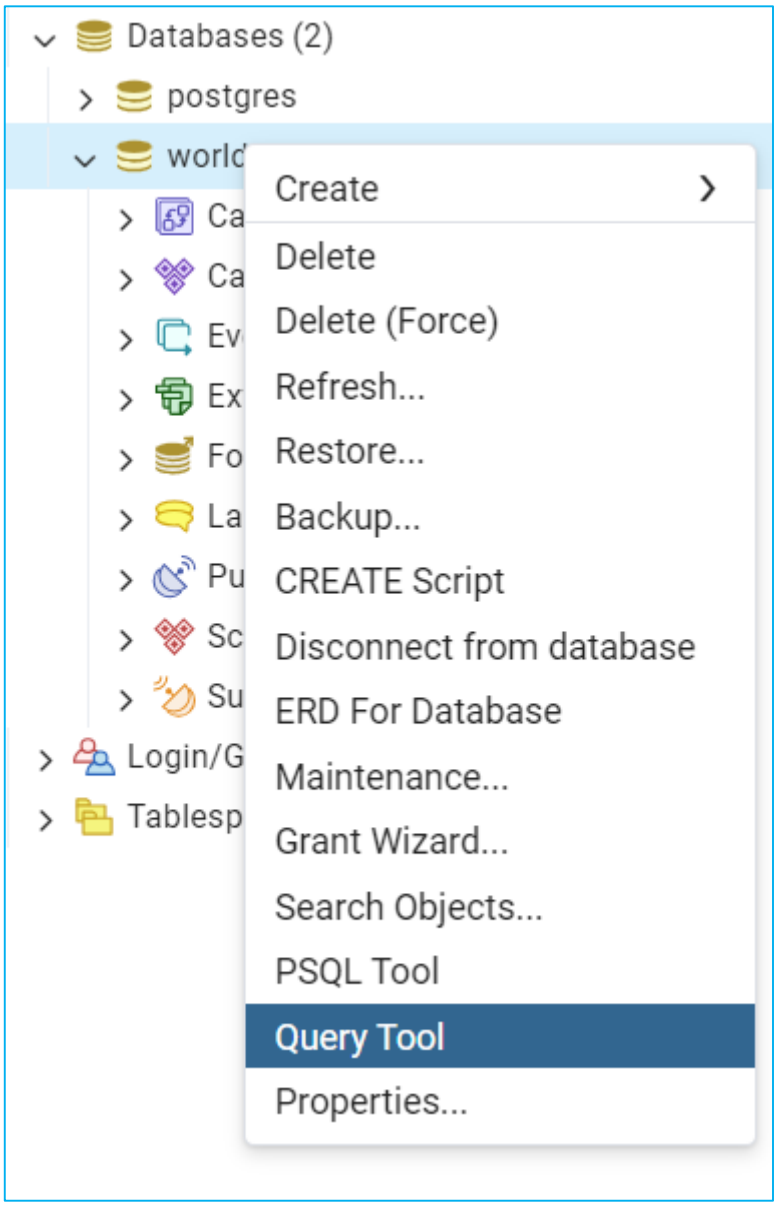


Para dotar a una base de datos de capacidades espaciales debemos ejecutar el comando ***Create extension*** en una base de datos para instalar la funcionalidad vectorial y raster.

Para ejecutar una consulta SQL debemos:

1. Seleccionar la base de datos *world*.

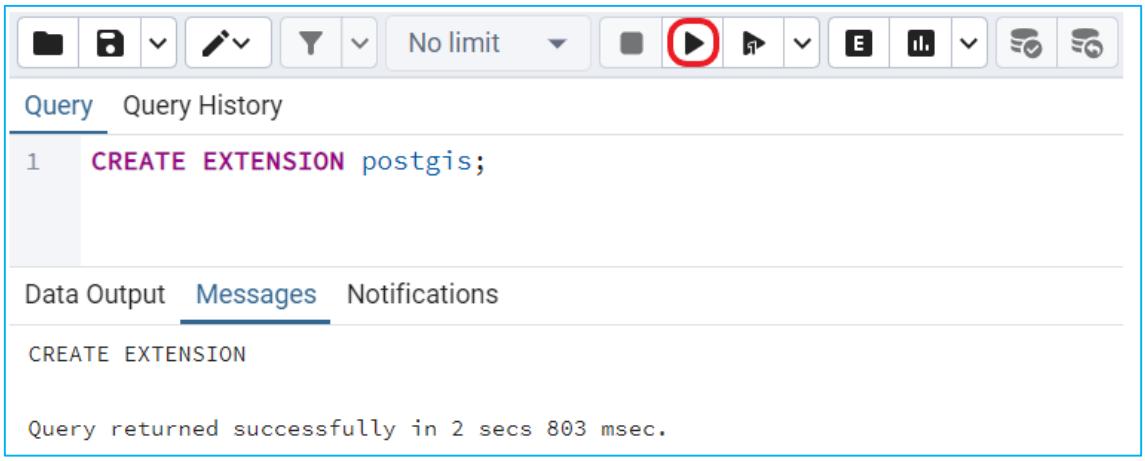
2. Hacer clic en el botón  o hacer clic con el botón derecho del ratón sobre la base de datos 'world' para abrir el cuadro de dialogo **Query Tool**.



Introduce la siguiente consulta en el Editor de consultas SQL:

```
CREATE EXTENSION postgis;
```

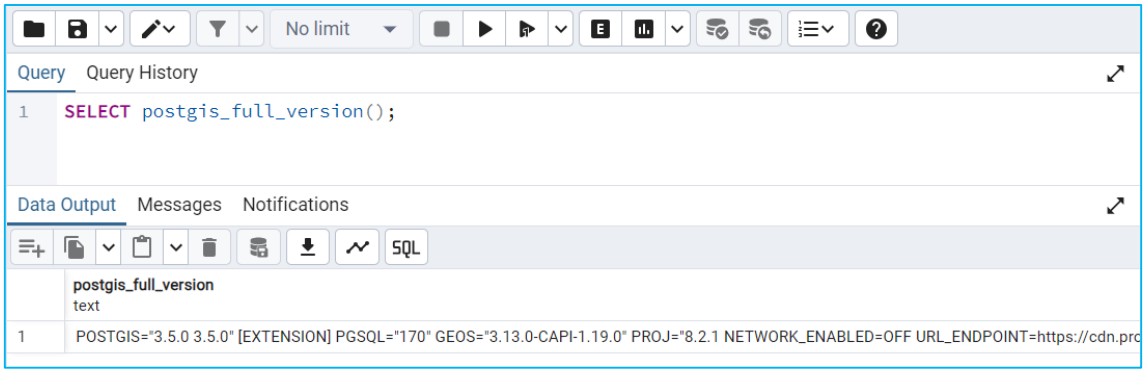
Haz clic sobre el botón *Execute/Refresh (F5)* para ejecutar la consulta.



Comprueba que la base de datos se creó correctamente con la función de gestión `postgis_full_version()` que nos devuelve la versión y la información sobre la configuración de esta compilación de PostGIS.

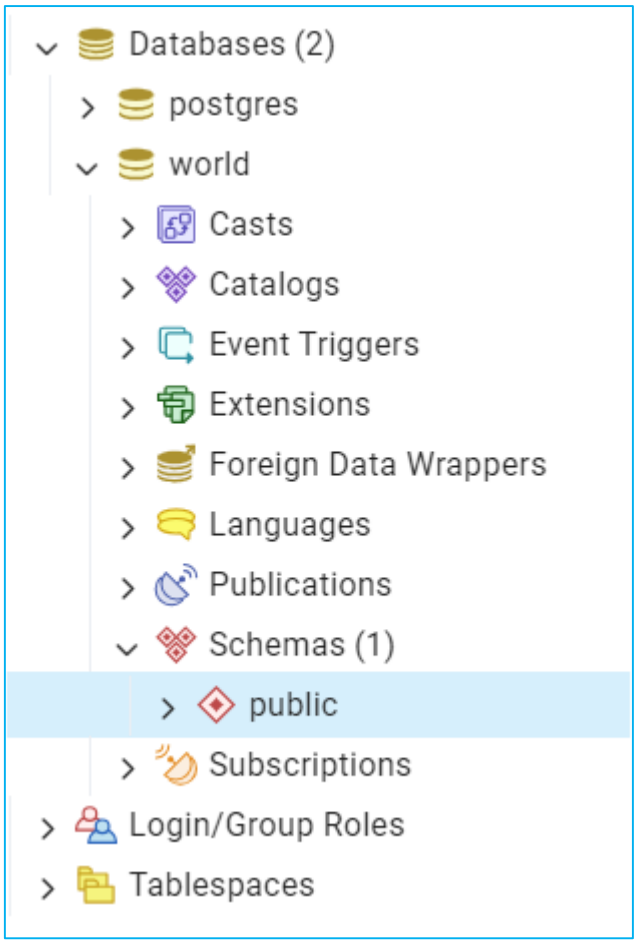
Elimina la sentencia anterior y escribe:

```
SELECT postgis_full_version();
```



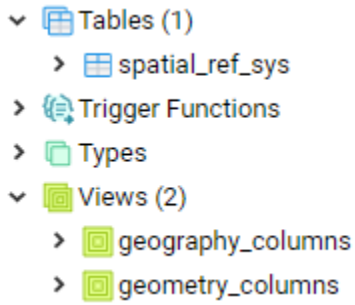
Cierra la ventana sin guardar los cambios.

Selecciona la nueva base de datos `world` (quizá necesites actualizar haciendo clic en F5) y ábrela para mostrar el árbol de objetos. Verás el esquema `public`:



Dentro de este esquema hay varios nodos. Los más importantes son dos:

- Un nodo denominado **Tables** que contiene una tabla de metadatos específica de PostGIS: `spatial_ref_sys`
- Un nodo denominado **Views** que contiene las vistas `geometry_colums`, `geography_colums`.



Creación de una base espacial desde la línea de comandos

También se puede crear una base de PostGIS desde línea de comandos con los comandos `createdb` y `psql`.

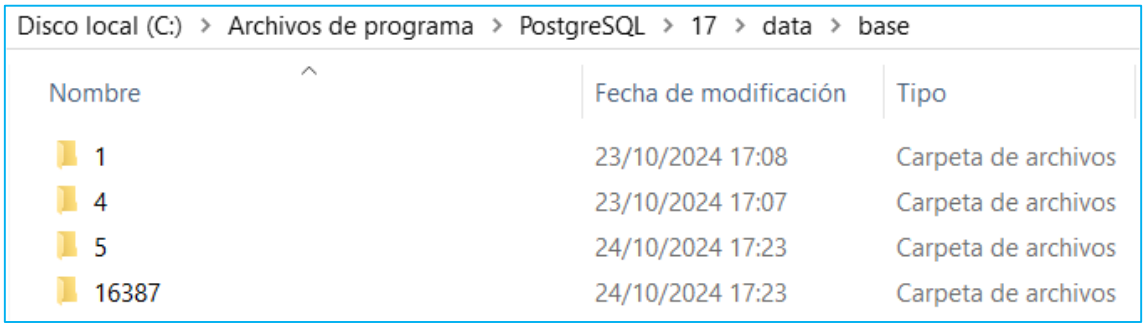
```
createdb <nombrebasededatos>
psql < nombrebasededatos > -c 'CREATE EXTENSION postgis'
```

¿Dónde guarda PostGIS las bases de datos?

Si investigamos un poco en el árbol de directorios donde está instalado PostgreSQL, podremos observar que donde guarda las bases de datos es en:

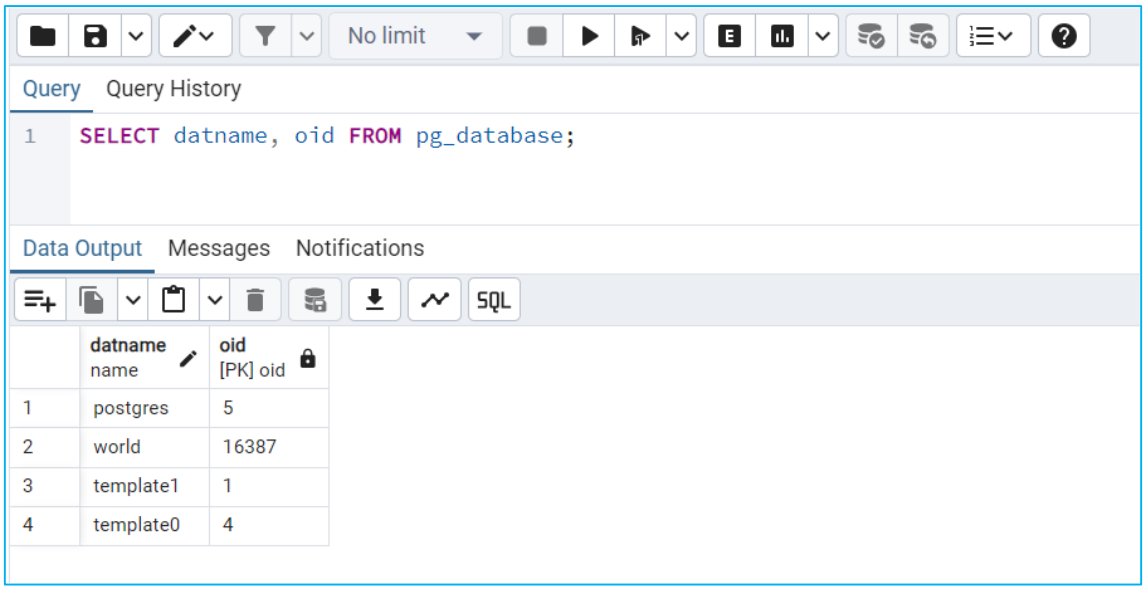
C:\Archivos de programa\PostgreSQL\17\data\base

Pero comprobamos que nuestras bases de datos no aparecen, o eso parece:



Como podemos observar en la imagen, en la carpeta *base* aparecen varias carpetas con números como nombre. Si queremos saber qué nombre de base de datos corresponden a esos números, es tan fácil como ejecutar esta consulta:

```
SELECT datname, oid FROM pg_database;
```



Como vemos en la imagen la carpeta con el número 16387 se corresponde con la base de datos *world*.

El esquema de tablas y vistas de PostGIS

Vamos a ver rápidamente las características principales de los elementos de la nueva base de datos PostGIS.

Nuestra nueva base de datos contiene automáticamente todas las funciones PostGIS, así como una tabla:

- spatial_ref_sys

y dos vistas:

- geometry_columns
- geography_colums

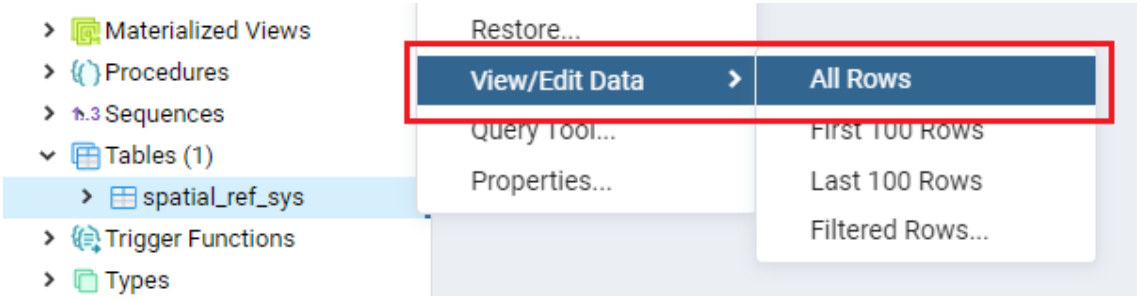
Tabla spatial_ref_sys

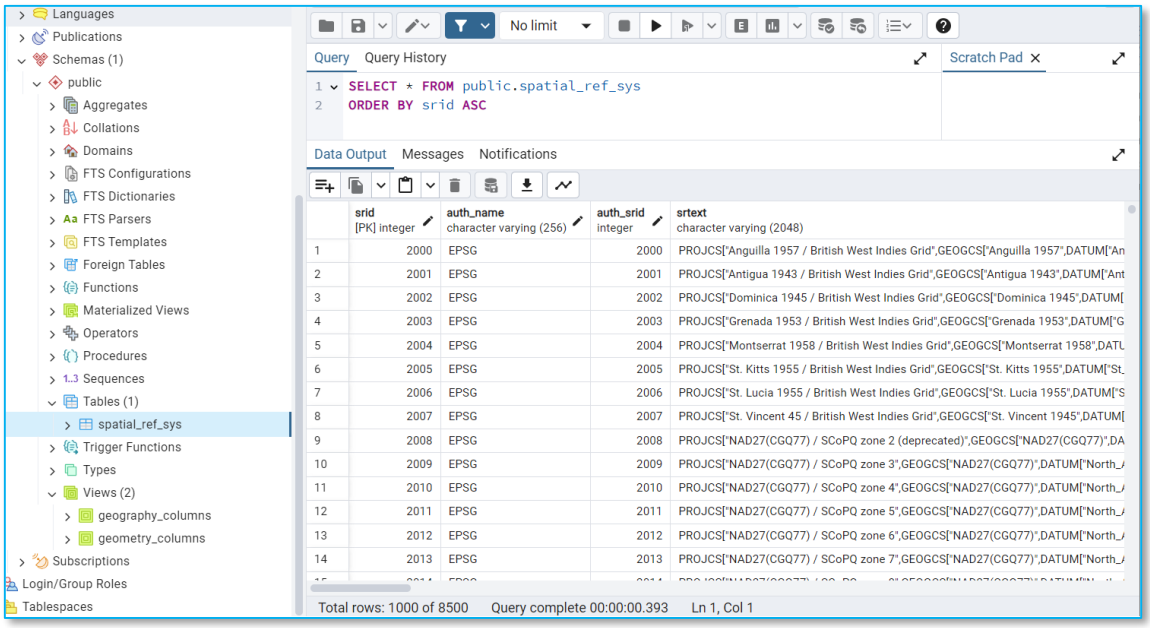
La tabla `spatial_ref_sys` define los sistemas de referencia espacial que se conocen de la base de datos. Son conocidos por un número de identificación, como 4326 (WGS 84 Lat / Lon).

Podemos ver los campos de las tablas y vistas pinchando sobre el elemento que acabamos de importar y haciendo clic sobre el botón del menú superior ***View Data***



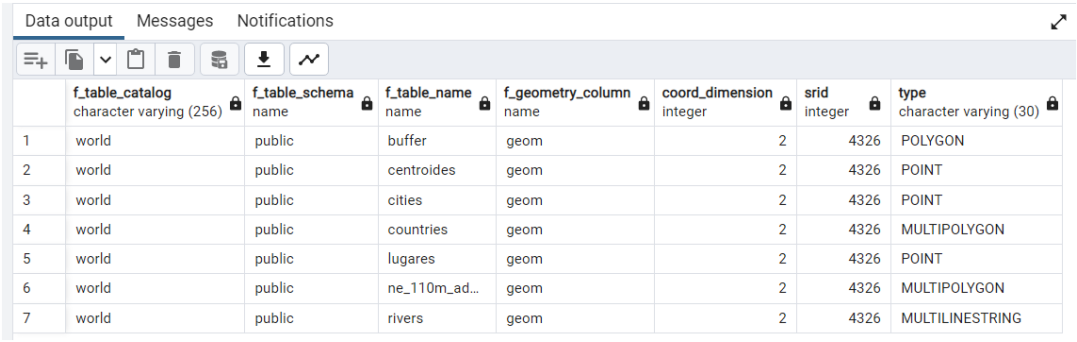
O bien en el menú superior ***Object > View/Edit Data > All Rows*** *para ver* los datos del objeto seleccionado:





Vistas geometry_columns y geography_columns

Las vistas `geometry_columns` y `geography_columns` definen la dimensión, geometría, y el sistema de referencia espacial para cada tabla PostGIS (si es espacial). Las **vistas se actualizan automáticamente** con los datos de las tablas creadas.



Data output | Messages | Notifications

	f_table_catalog character varying (256)	f_table_schema name	f_table_name name	f_geometry_column name	coord_dimension integer	srid integer	type character varying (30)
1	world	public	buffer	geom	2	4326	POLYGON
2	world	public	centroides	geom	2	4326	POINT
3	world	public	cities	geom	2	4326	POINT
4	world	public	countries	geom	2	4326	MULTIPOLYGON
5	world	public	lugares	geom	2	4326	POINT
6	world	public	ne_110m_ad...	geom	2	4326	MULTIPOLYGON
7	world	public	rivers	geom	2	4326	MULTILINESTRING

Esta imagen es ilustrativa, no deberías ver todavía ningún elemento en la vista.

La importación y exportación de datos en PostGIS

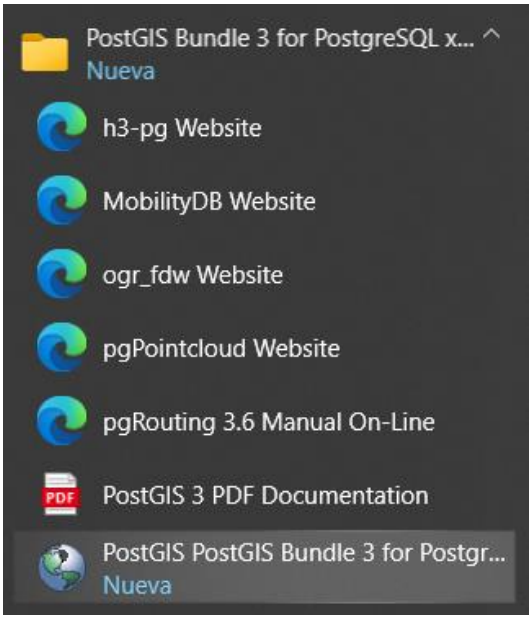
Abordamos ahora una cuestión básica como es la de incorporar los datos GIS a nuestra base de datos. Hay varias formas:

- Mediante la herramienta shp2pgsql, que se invoca desde la consola de comandos. Tiene el inconveniente de que solo sirve para importar los archivos shapefile de ESRI.
- **La versión gráfica del importador shp2pgsql** (*PostGIS shapefile Import/Export Manager*). Esta versión es mucho más amigable y es la que utilizaremos.
- **Mediante el uso de QGIS y su administrador de base de datos.** Este sistema tiene la ventaja de que nos permite incorporar a PostGIS datos en otros formatos alternativos al shapefile como es el caso de GeoPackage.

La herramienta *PostGIS 3 Shapefile and DBF Loader Exporter*

La herramienta ***PostGIS 3 Shapefile and DBF Loader Exporter*** nos permite importar y/o exportar *shapefiles* **o archivos DBF** a la base de datos.

1. Para abrir la aplicación deberás ir a inicio > Todos los programas > PostGIS Bundle 3 for PostgresSQL > **PostGIS 3 Shapefile and DBF Loader.**



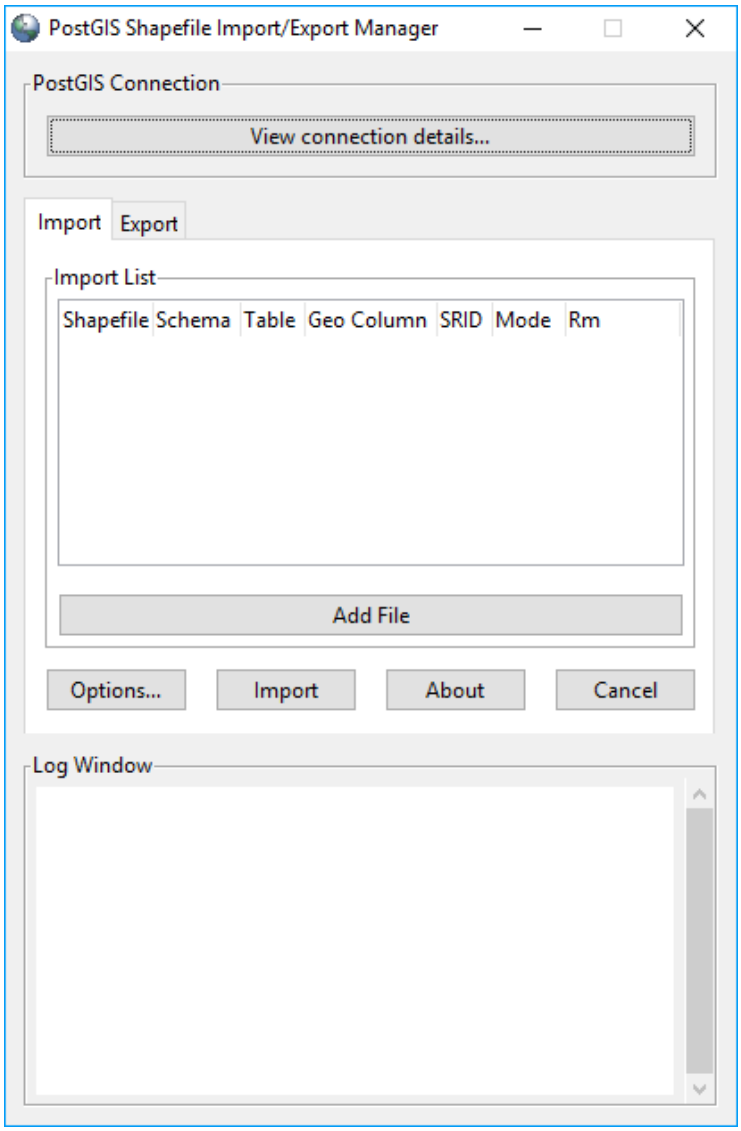
Nota

Si no puedes abrir **PostGIS 3 Shapefile and DBF Loader** debes crear una variable de entorno en el PATH con la ruta: C:\PostgreSQL\17\bin

Debéis ir a Configuración avanzada del sistema > Variables de entorno. En las **variables del sistema** buscar la variable Path y pinchar sobre **Editar...**

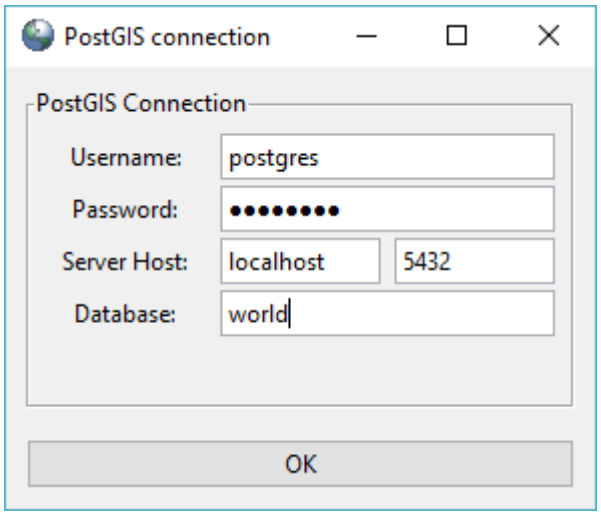
Añadir la nueva ruta C:\PostgreSQL\17\bin y aceptar todos los cuadros de diálogo.

2. La interfaz se pondrá en marcha.

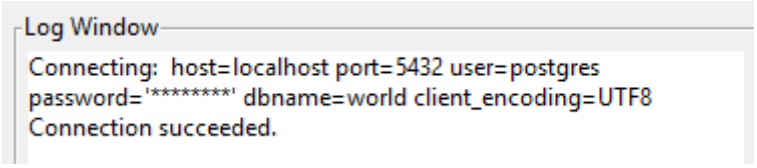


3. Pincha en el botón "*View connection details*..." y rellena la información solicitada con los datos de la tabla, cuando lo tengas haz clic en **OK**.

Username	`postgres`
Password	`postgres`
Server Host	`Localhost 5432`
Database	`world`

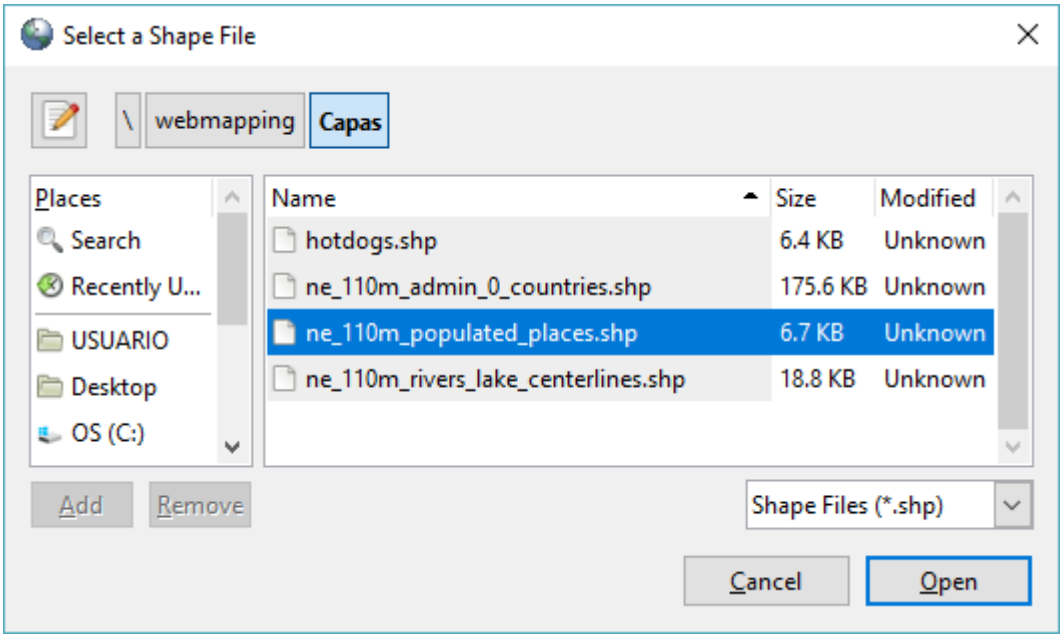


4. Un mensaje nos indicará que la conexión se ha realizado con éxito.



5. Abre el explorador de *shapefiles* asegurándote de que la pestaña ***Import*** está seleccionada y haz clic en el botón ***Add File.***

6. Navega hasta el directorio `<webmapping>\Capas`. Selecciona el archivo *ne_110m_populated_places.shp*.

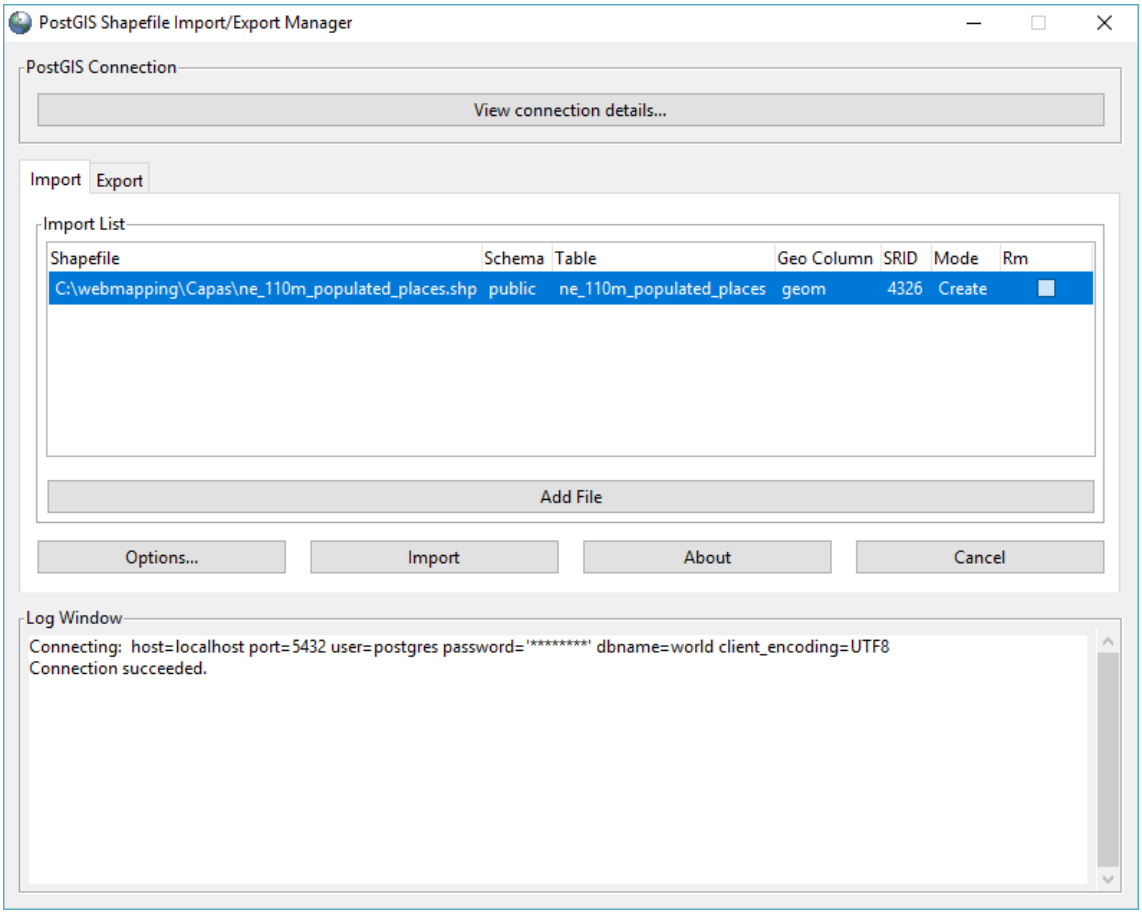


7. A continuación, rellena los datos para cada elemento de la lista de importación con especial atención en proporcionar el **SRID correcto** para cada shapefile a 4326 (de 0)

schema	`public`
SRID	`4326`
geo column	`geom`
mode	`Create`

Importante

No omitas el cambio de SRID, de lo contrario los datos no se cargarán correctamente.

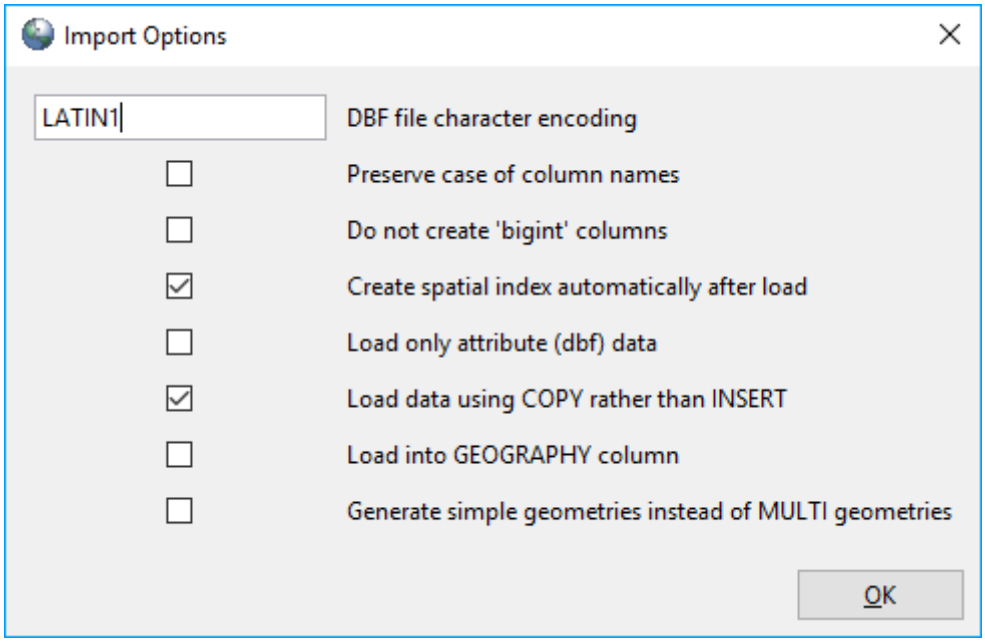


8. Vamos a editar la información de configuración. Haz clic en el botón ***Options...*** y selecciona:

    1. LATIN1 para codificar los caracteres.

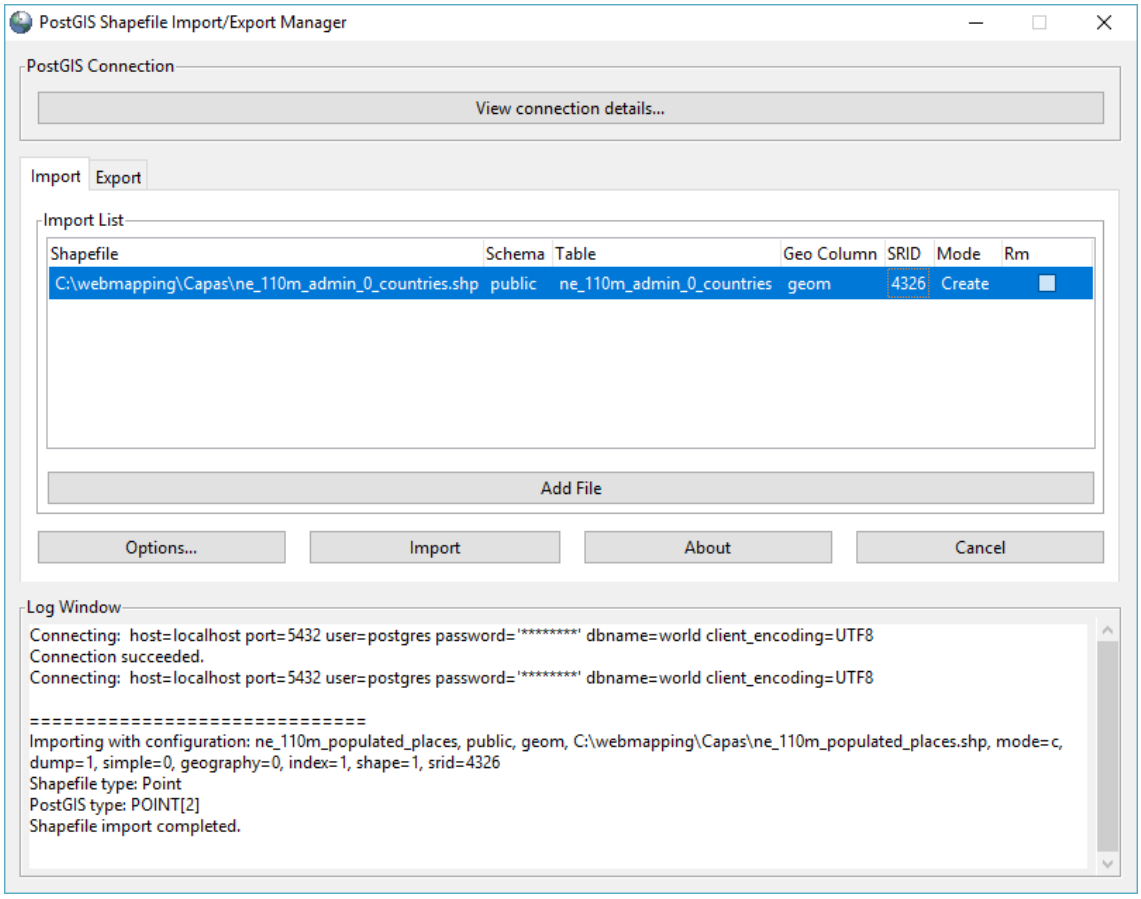
    2. *Load data using COPY rather than INSERT*. Esto hará que el proceso de carga de datos sea más rápido.

    3. La opción *Load into GEOGRAPHY column* habilita las funciones geográficas. Nosotros trabajaremos con las funciones geométricas, por lo tanto, **no marques** esta casilla.



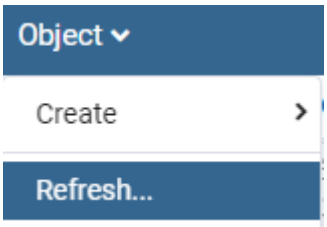
9. Clic en **OK** y, por último, pincha sobre ***Import*** para iniciar el proceso de importación.

10. Repite el proceso de importación para el shapefile *ne_110m_admin_0_countries.shp*. Puedes eliminar la capa anterior de la lista haciendo clic sobre casilla situada bajo **Rm**.

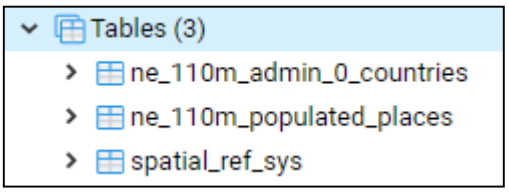


2. Cuando los *shapefiles* se hayan cargado, podemos cerrar esta aplicación y volver a pgAdmin.

3. Sitúate sobre el nodo Tables, ve al menú *Object* y haz clic en el botón "***Refresh...***" de este modo se actualizará la vista del árbol.



4. Deberías ver dos nuevas tablas en **Databases > world > Schemas > public > Tables.**



Conexión desde un SIG a la base de datos PostGIS

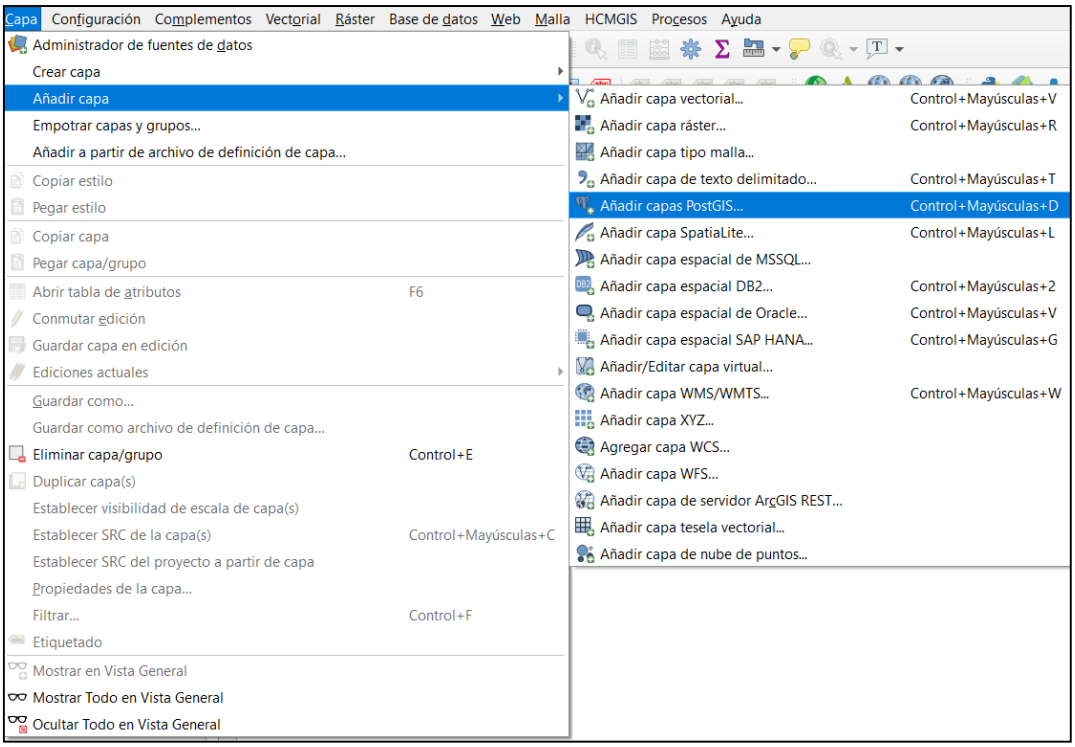
Muchas aplicaciones pueden conectarse a una base de datos PostGIS, y visualizar y editar datos de una manera más atractiva.

No es el objeto de este curso instalar un cliente SIG en tu ordenador (seguro que ya tienes alguno instalado ;-)), pero será necesario para visualizar y añadir los datos a PostGIS.

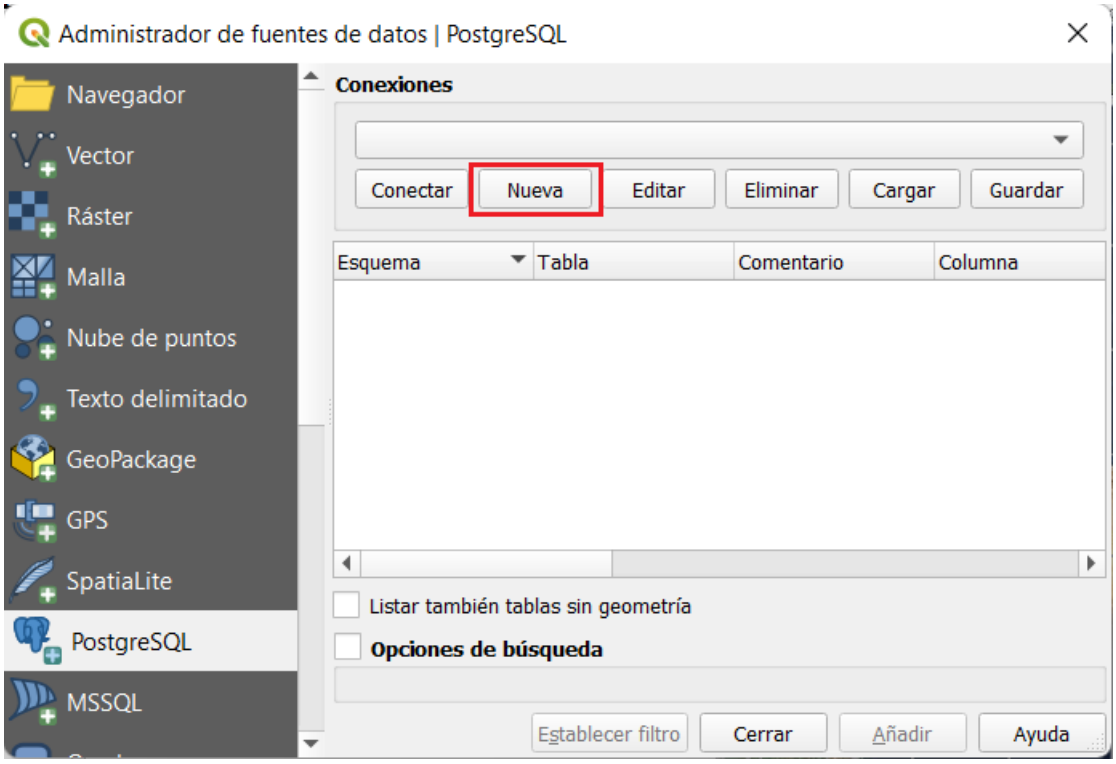
Explicaremos aquí brevemente la **conexión a PostGIS desde QGIS**, porque luego lo utilizaremos para subir datos vectoriales a nuestra base de datos empleando el administrador de BBDD de QGIS.

Para realizar la conexión desde QGIS seguimos el siguiente itinerario:

Abrimos QGIS y en el menú superior **Capa**, seleccionamos **Añadir Capa** > **Añadir capas PostGIS:**



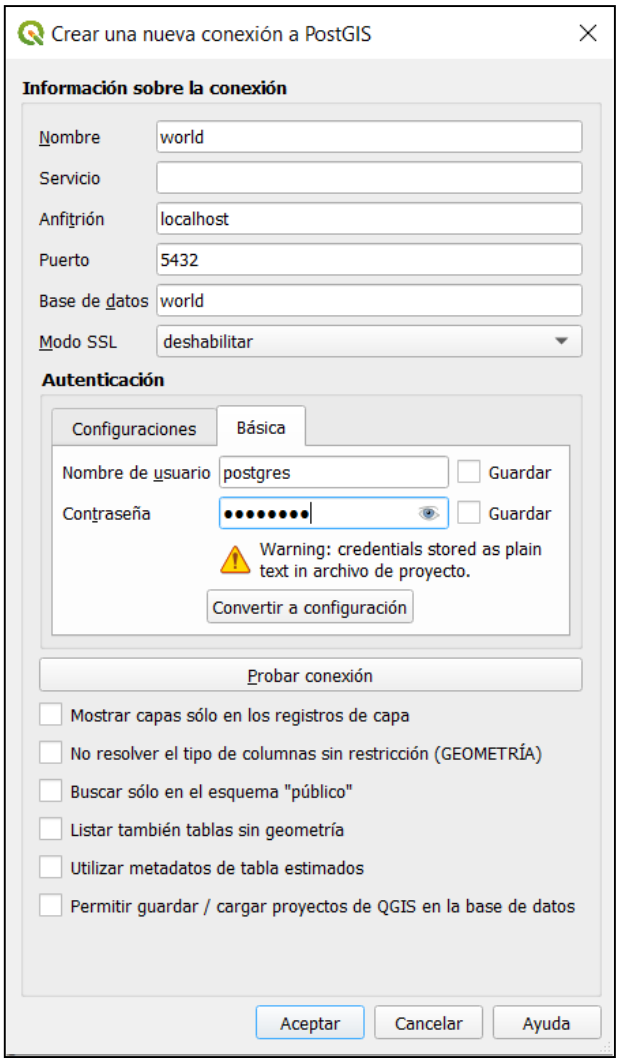
En la ventana de conexiones que se abre hacemos clic en **Nueva:**



Nos pide los datos para la conexión.

- **Nombre**: establecemos un nombre que defina a la conexión. Podemos poner el que queramos pero pondremos ***world*** para reconocer la conexión a las capas del mundo.
- **Servicio:** Lo dejaremos en blanco. El parámetro Servicio permite especificar un archivo de conexión, que contiene los parámetros necesarios para acceder a una determinada base de datos. Si empleásemos este archivo podríamos omitir la entrada de puerto, usuario o contraseña, dejándolos en blanco.
- **Anfitrión**: el nombre del servidor.
- **Puerto:** normalmente será 5432.
- **Base de datos**: Nombre de la base de datos donde tenemos las capas que queremos añadir a QGIS. En este caso *world*.
- En el apartado ***Autenticación*** debemos ir a la pestaña ***Básica*** e introducir el nombre de usuario y contraseña: en PostgreSQL por defecto serán postgres / postgres.

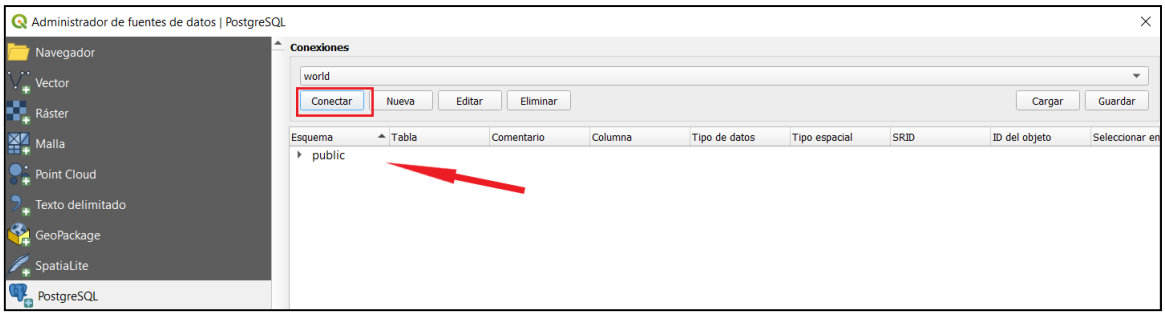
Los parámetros de conexión son:



Username	postgres
Password	postgres
Server	Localhost
Port	5432
Database	world

Una vez creada la conexión, solamente tenemos que hacer clic sobre el botón **Conectar** y si hemos escrito bien los parámetros, debe realizarse la conexión.

Nos aparecerá el esquema ***public***. Y al hacer clic sobre ***public*** veremos las bases de datos con sus tablas.

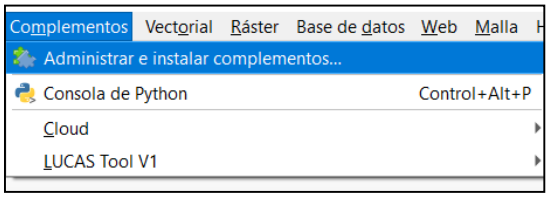


Ejercicio nº3. Realiza una conexión desde un cliente SIG de escritorio a la base de datos world y pega el resultado en la plantilla de resultados.

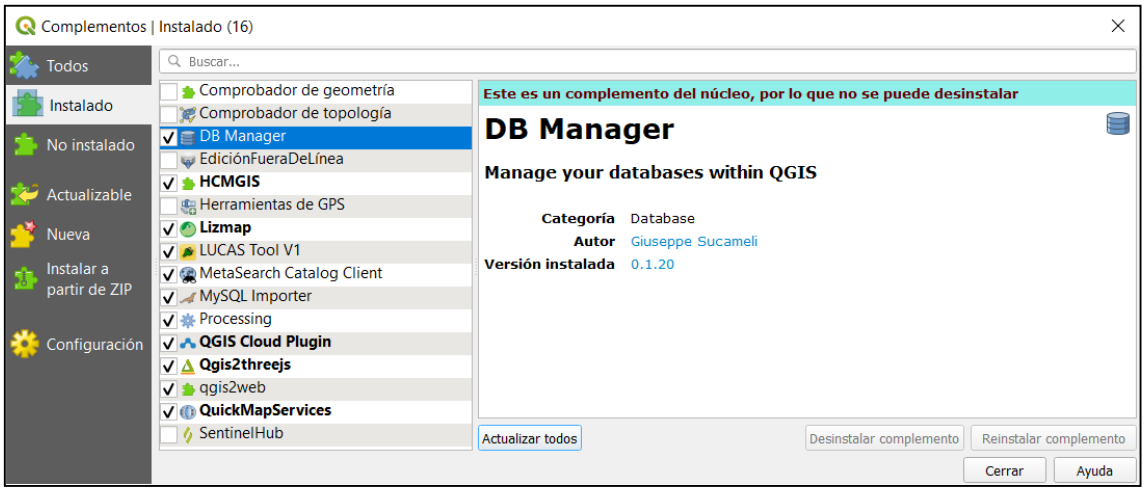
Importar cartografía a PostGIS con el administrador de BBDD de QGIS

El administrador de BBDD de QGIS o DB Manager es un complemento del núcleo de QGIS, por lo que ya lo tendrás instalado. Sirve para gestionar las bases de datos GeoPackage, Oracle Spatial, PostGIS o SpatialLite.

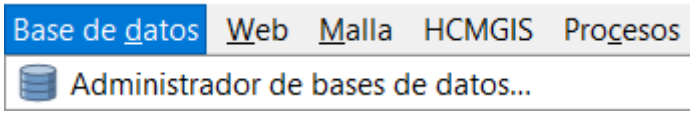
En el menú "Complementos" de QGIS hacemos clic en **Administrar e instalar complementos**.



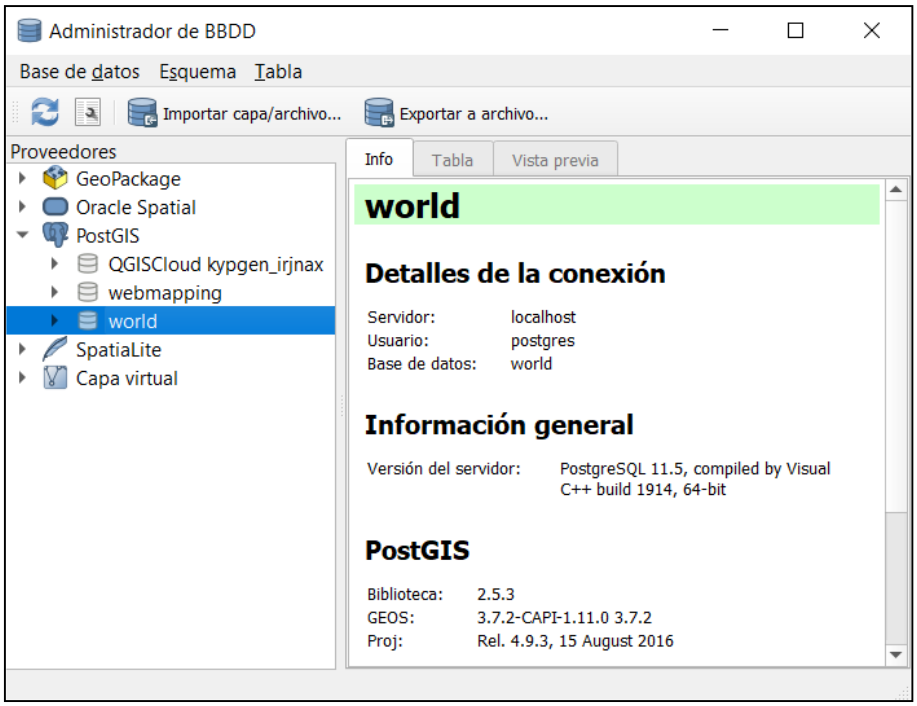
Aquí veremos el complemento DB Manager, marca la casilla de verificación si no la tienes marcada:



Para abrirlo, vamos al menú superior de QGIS, **Base de datos > Administrador de bases de datos...**

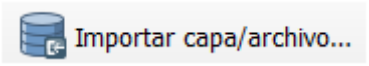


Al abrir el Administrador de bases de datos nos aparecerán las conexiones que tenemos. Entre ellas debe estar la que creamos antes:



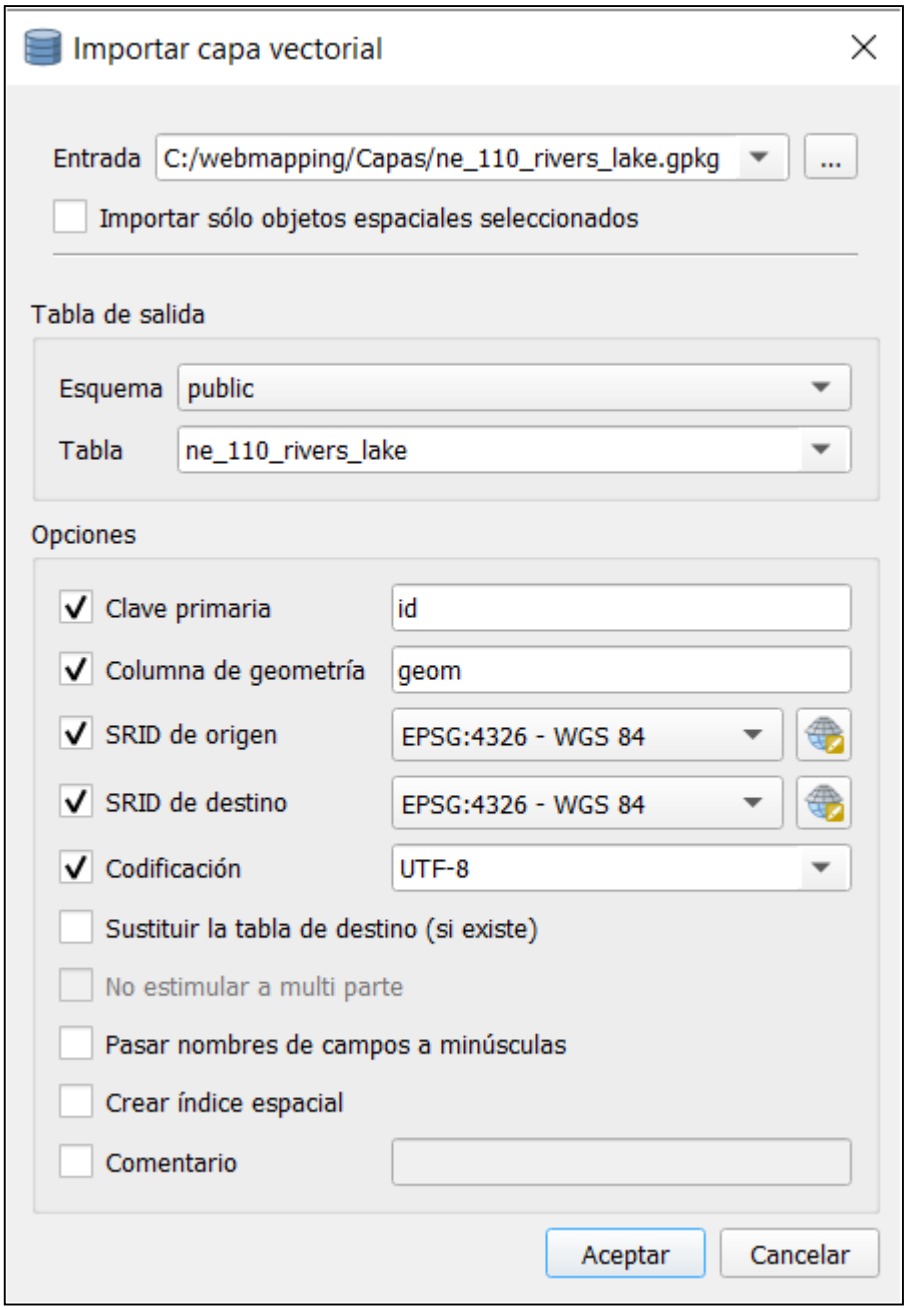
Ahora ya podemos importar los archivos GeoPackage, shapefile o GeoJSON a nuestra base de datos PostGIS.

Debemos hacer clic en el botón **Importar capa/archivo...**

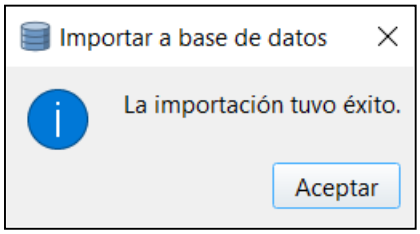


A continuación, se abrirá el cuadro de diálogo «**Importar capa vectorial**».

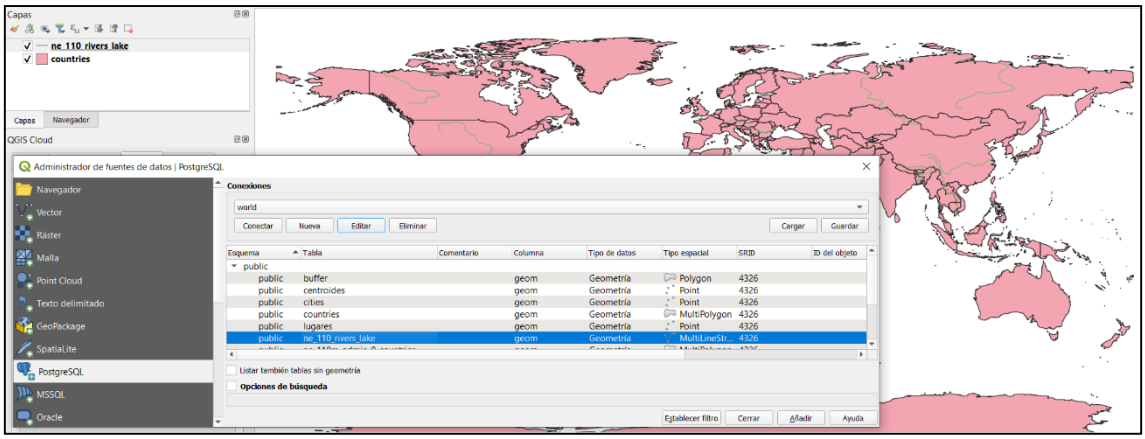
- **Entrada:** Seleccionamos la capa en el formato vectorial elegido, en nuestro caso seleccionamos el archivo **ne_110_rivers_lake.gpkg**, que lo puedes encontrar en la carpeta de archivos del curso. Como puedes observar se trata de un archivo en formato GeoPackage.
- **Esquema**: *public*
- **Tabla**: Indicamos el nombre para la tabla: **rivers**.
- **Clave primaria**: id.
- **Columna de geometría**: Deja el nombre por defecto, geom.
- **SRID de origen**: EPSG:4326 – WGS 84.
- **SRID de destino**: EPSG:4326 – WGS 84.
- **Codificación**: UTF-8.



Nos indicará que la importación se ha producido con éxito.



Puedes comprobar que efectivamente se ha creado la tabla abriendo de nuevo la conexión a PostGIS e incorporando la capa.

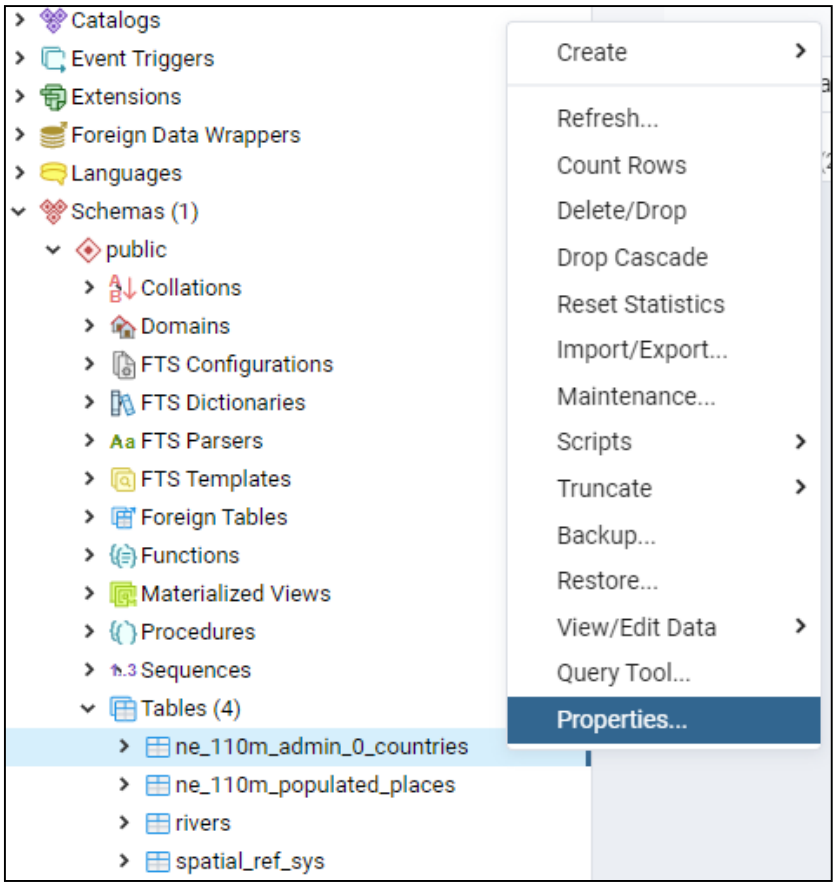


Ahora que hemos creado las tablas en nuestra base de datos haz las siguientes operaciones:

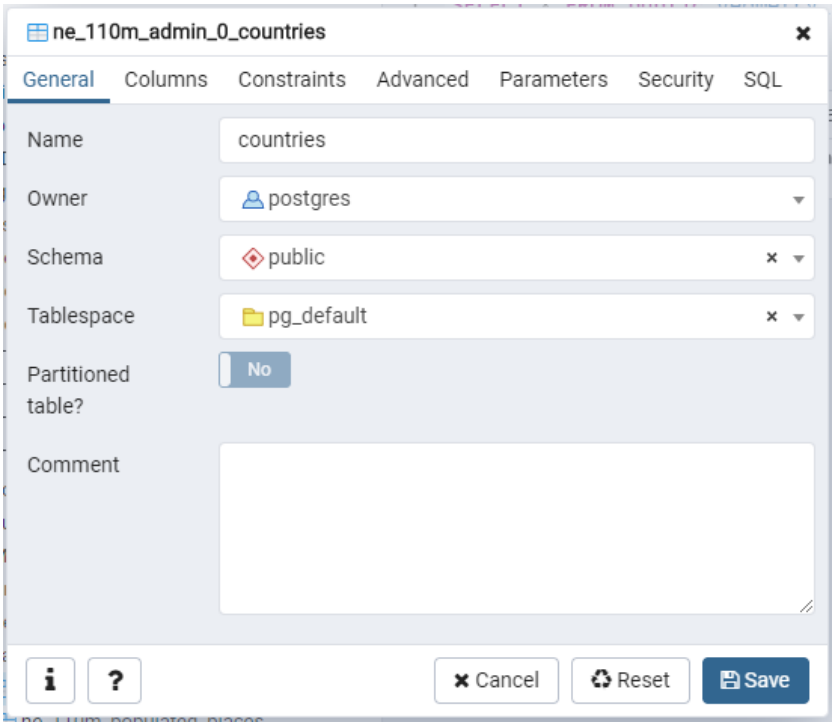
1. Si entras en pgAdmin 4, encontrarás la nueva tabla dentro de la base de datos world > Schemas > public > Tables.

2. Para facilitar el trabajo con estas tablas vamos renombrar las tablas *ne_110m_admin_0_countries* y *ne_110m_populated_places* a *countries* y *cities*, respectivamente.

3. Para ello hacemos clic derecho sobre cada una de las tablas > *Properties*...



En la primera pestaña (***Properties***), modificamos los nombres de los campos en el apartado *Name*.



Clic en **Save**.

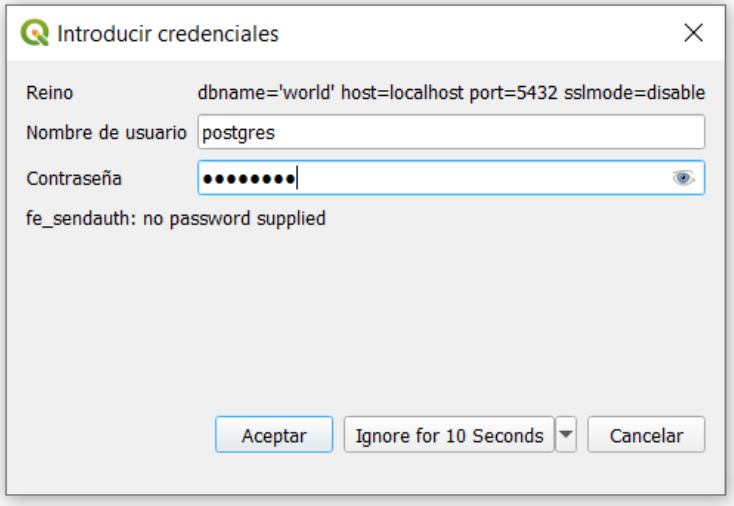
Realizamos la misma operación para la tabla *ne_110m_populated_places* modificando el nombre a ***cities***

Ejercicio nº 4. Haz una captura de pantalla que muestre las 3 tablas creadas (dos de ellas con la interfaz gráfica y otra con el Administrador de bases de datos de QGIS)

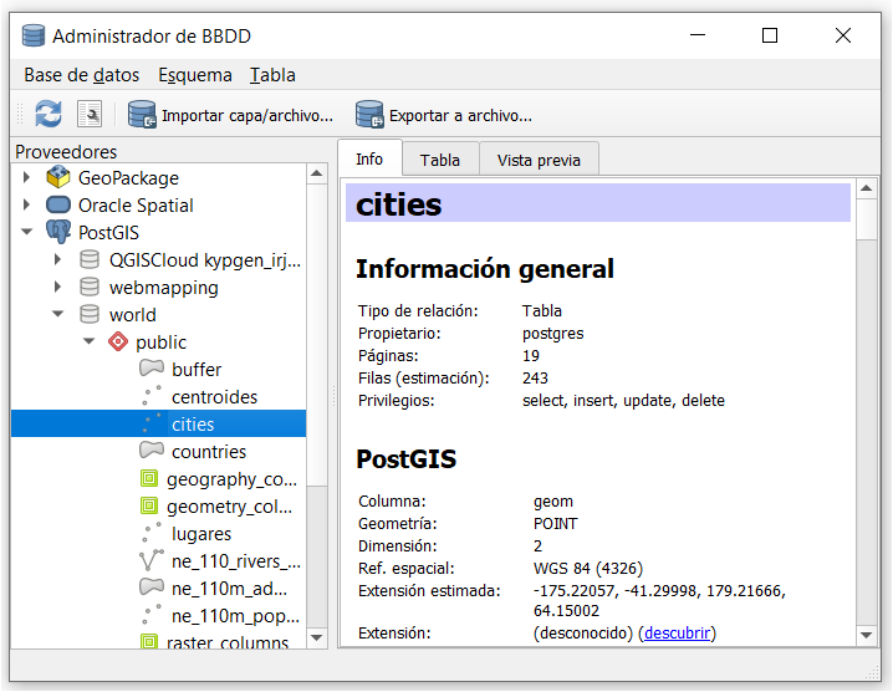
Exportar tablas de PostGIS a archivos vectoriales con el administrador de BBDD de QGIS

Desde el administrador de bases de datos de PostGIS también podemos exportar las tablas, con la ventaja de que nos permite hacerlo en formatos como: GeoPackage, Archivo Shape de ESRI, AutoCAD DXF, GeoJSON y muchos más.

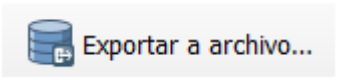
Si has retomado el trabajo desde aquí puede que te pida ingresar las credenciales: postgres/postgres



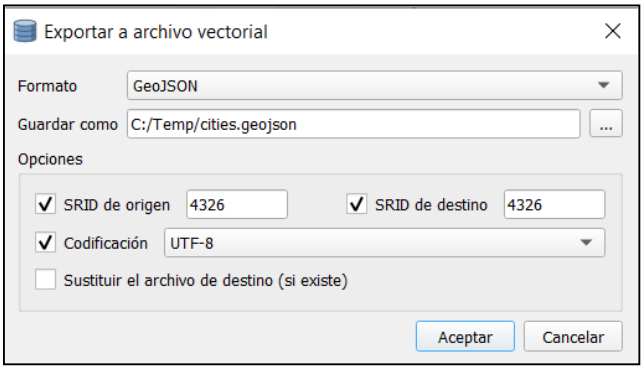
Cuando tengamos abierto el administrador de BBDD, seleccionamos PostGIS y la base de Datos. A continuación, seleccionamos la tabla a exportar. Haz clic en *cities* para seleccionar esta tabla.



Cuando tengas la tabla seleccionada haz clic sobre el botón: **Exportar a archivo**



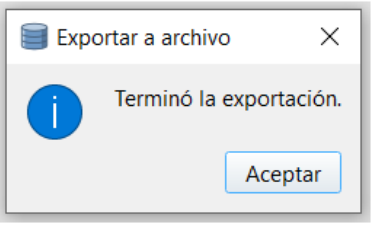
Se nos abre una ventana donde debemos escribir los datos para la exportación del archivo.



Completamos el formulario:

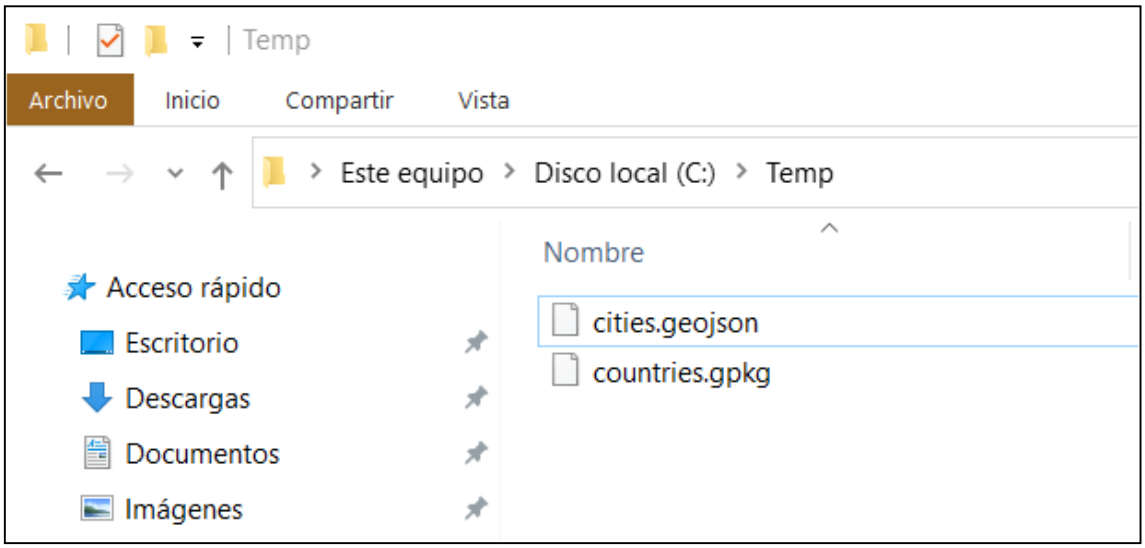
- **Formato:** nos permite elegir el formato de salida, Selecciona GeoJSON.
- **Guardar como:** indica la ruta, puedes elegir el directorio Temp en C: o cualquier otro si lo prefieres.
- **Opciones**: nos permite elegir el SRID de Origen y de salida, así como la codificación. Indicamos 4326 tanto para el SRID de entrada como para el de salida. En Codificación: UTF-8.

QGIS nos ofrece un mensaje indicando que se ha exportado el archivo.



Ahora dispones de los datos de la tabla *cities* en un formato vectorial que puedes utilizar en un mapa de Leaflet u Openlayers.

1. Vamos a la carpeta, y encontramos los archivos que hemos creado.



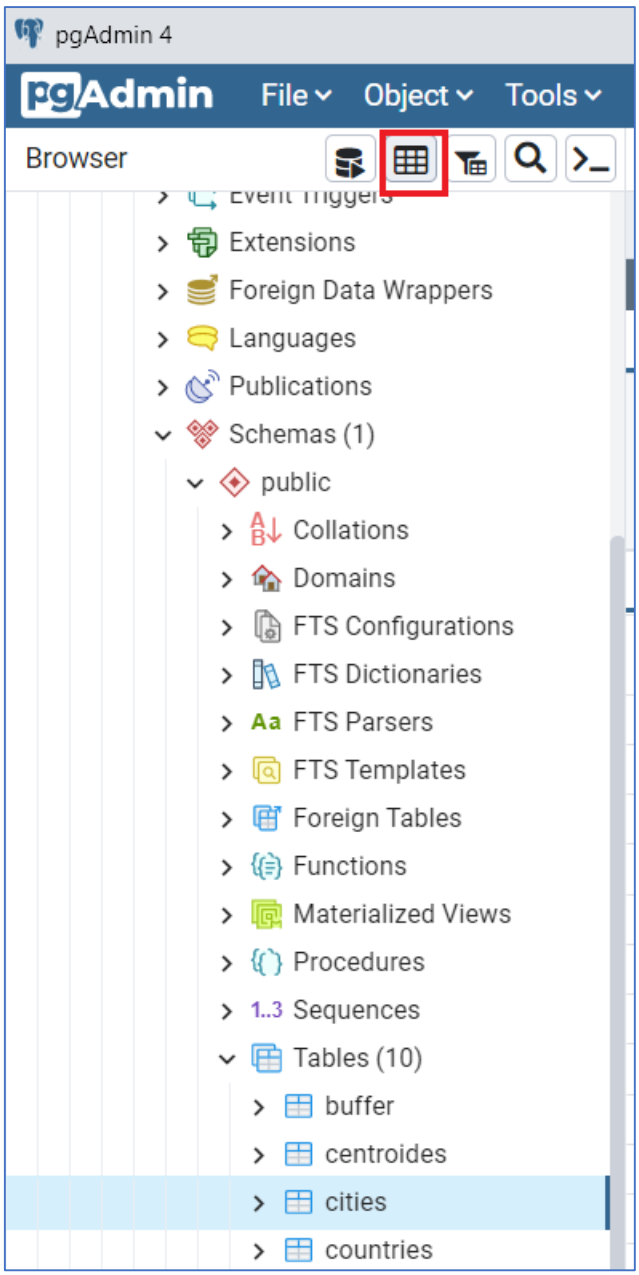
Ejercicio nº5. Haz una captura de pantalla como la de arriba en que se vean los dos archivos exportados (uno en formato GeoJSON y el otro en formato GeoPackage) y pégala en la plantilla de resultados.

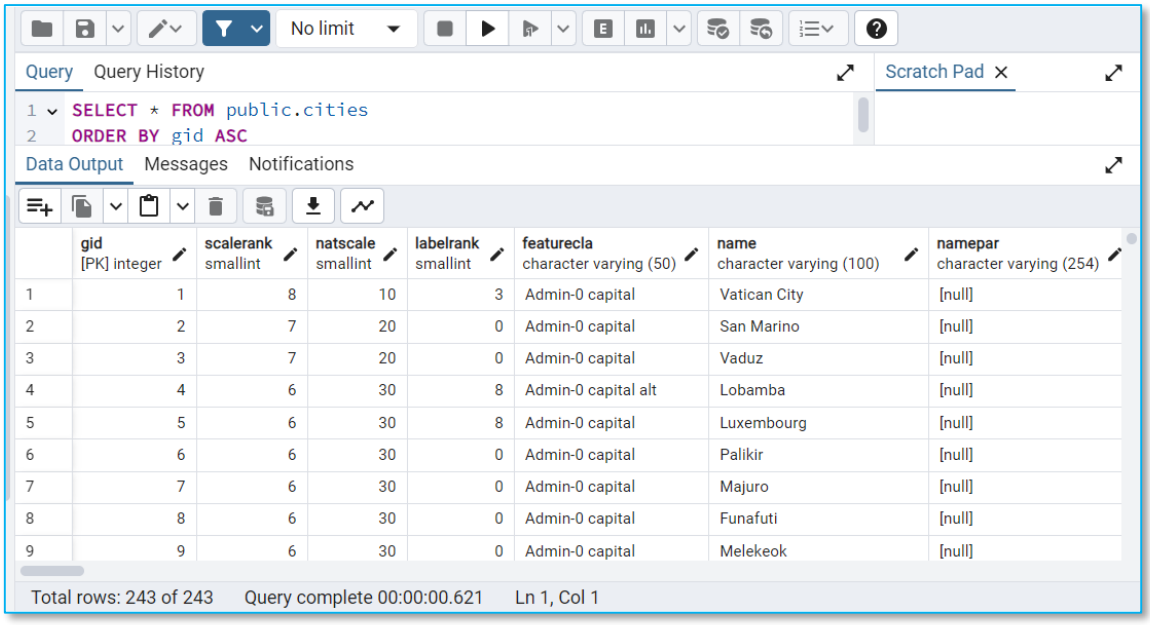
Visualización de los datos espaciales en PostGIS

El visor de geometrías integrado en pgAdmin

Las capas en PostGIS aparecen como tablas con un campo de geometría.

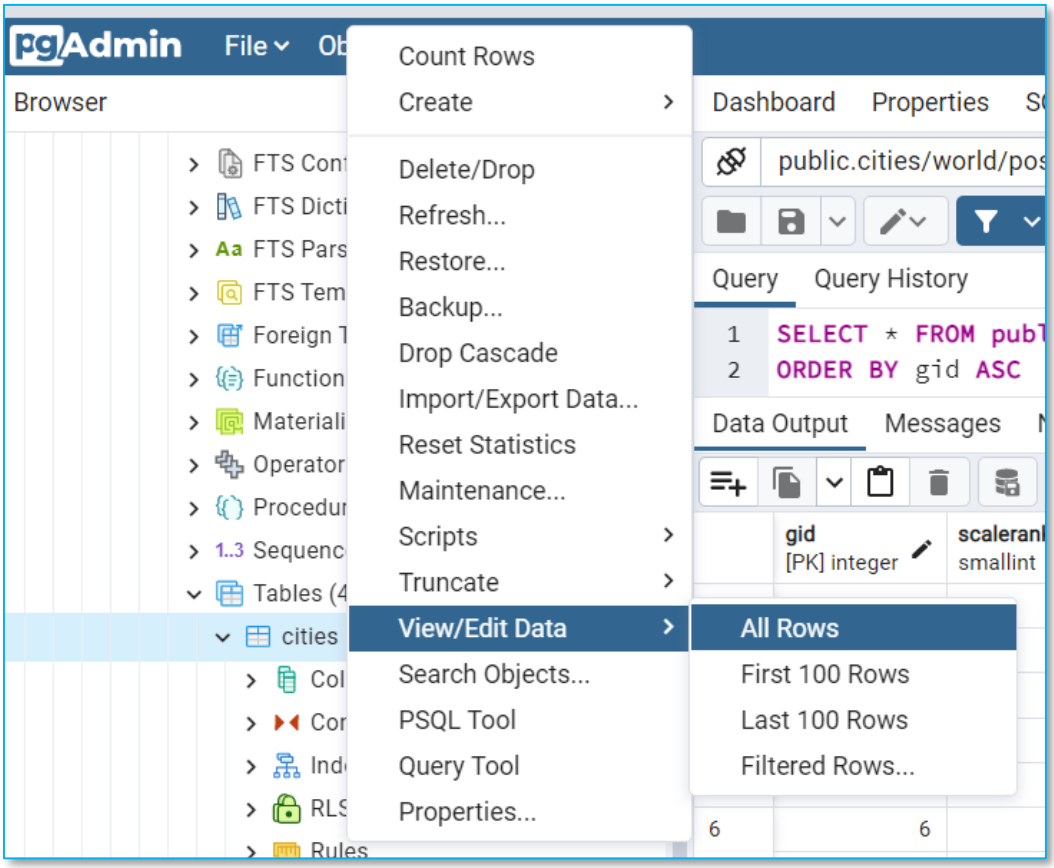
Podemos ver los campos de la tabla haciendo clic con el botón derecho sobre las capas que acabamos de importar y seleccionar la opción ***View Data***:



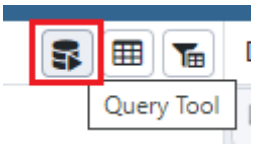


	gid [PK] integer	scalerank smallint	natscale smallint	labelrank smallint	featurecla character varying (50)	name character varying (100)	namepar character varying (254)
1	1	8	10	3	Admin-0 capital	Vatican City	[null]
2	2	7	20	0	Admin-0 capital	San Marino	[null]
3	3	7	20	0	Admin-0 capital	Vaduz	[null]
4	4	6	30	8	Admin-0 capital alt	Lobamba	[null]
5	5	6	30	8	Admin-0 capital	Luxembourg	[null]
6	6	6	30	0	Admin-0 capital	Palikir	[null]
7	7	6	30	0	Admin-0 capital	Majuro	[null]
8	8	6	30	0	Admin-0 capital	Funafuti	[null]
9	9	6	30	0	Admin-0 capital	Melekeok	[null]

También podemos hacer clic con el botón derecho del ratón sobre la tabla y seleccionar en el menú: **View/Edit Data** y a continuación el número de filas a mostrar.

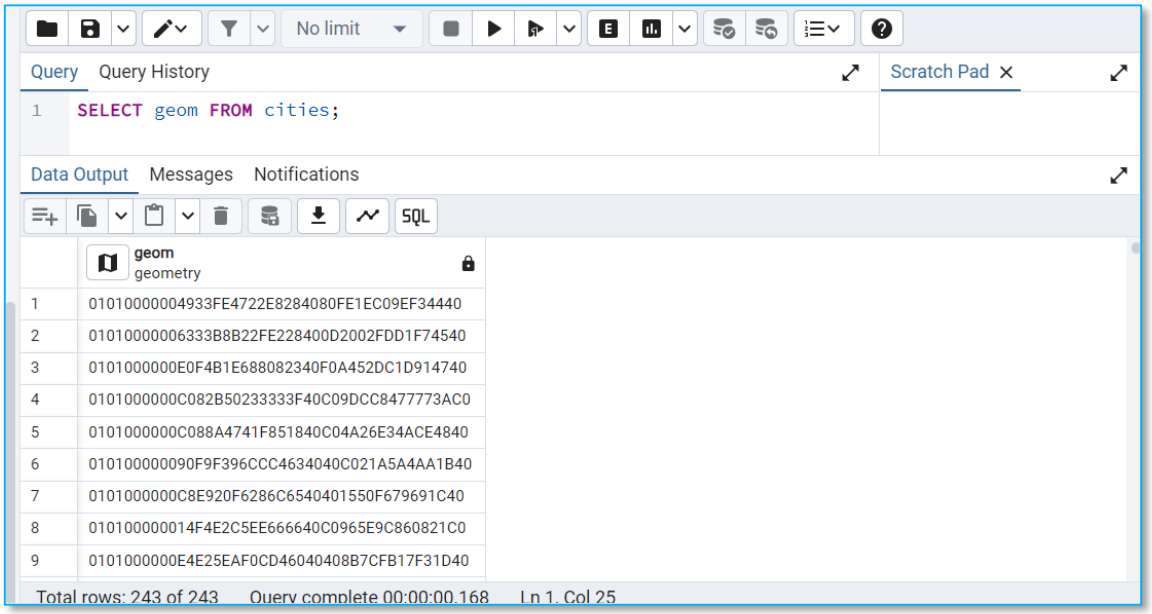


Para ver únicamente el campo geometría de una tabla podemos ejecutar una consulta SQL.



y escribir:

```
SELECT geom FROM cities;
```



Sin embargo, este archivo "binario" no es legible por nosotros. Por lo tanto, necesitamos algo para ayudarnos a interpretar y ver nuestros datos PostGIS.

Pero estamos de suerte, pgAdmin incluye un **visor de geometrías integrado**.

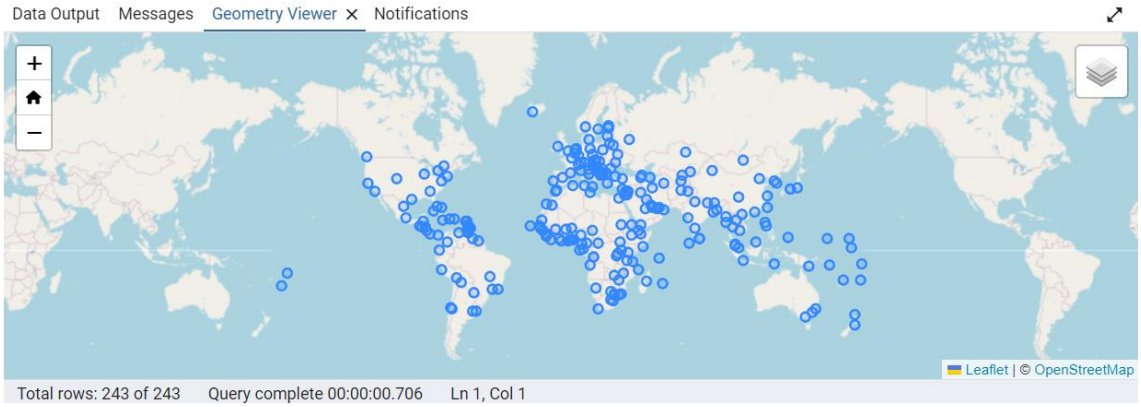
Con este visor integrado podremos ver los resultados de nuestras consultas directamente en pgAdmin, siempre y cuando el resultado de la consulta genere una columna de *geometry* o *geographic* (columnas *geom* o *geog*).

Si la geometría se encuentra en el SRID 4326 (WGS 84 lon / lat), pgAdmin las mostrará automáticamente sobre un mapa base de OpenStreetMap.

En la columna ***geom*** es donde se encuentra el botón "View all geometries in this column":



Al hacer clic en el botón se abrirá el mapa:

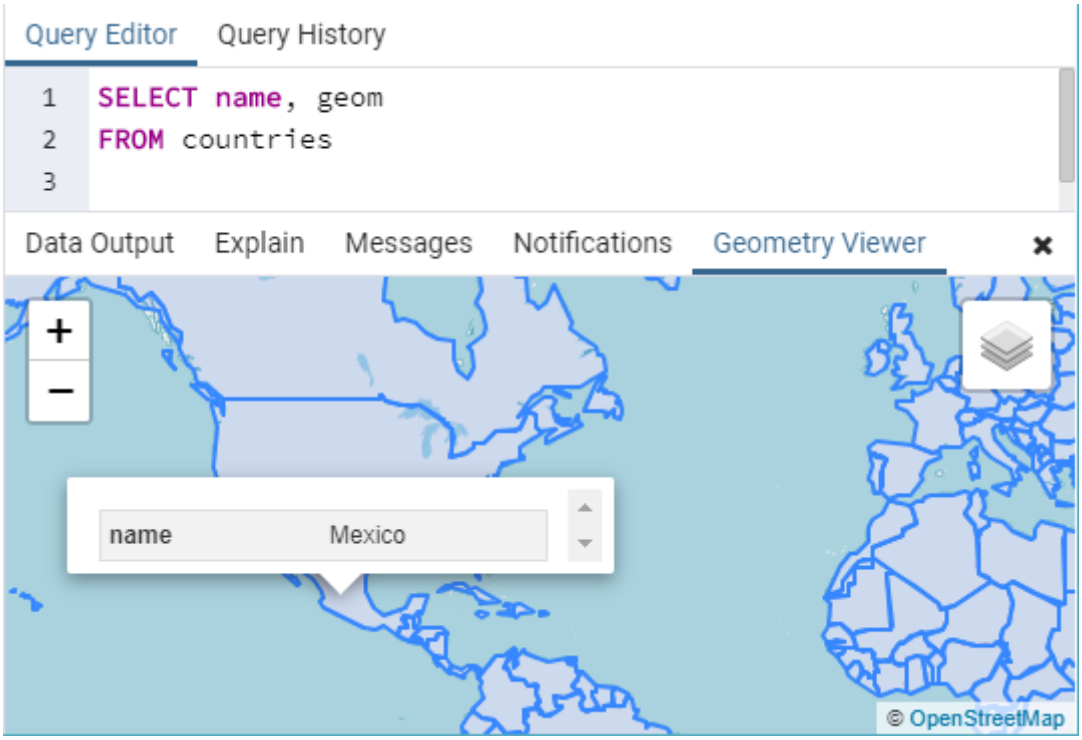


El visor muestra por defecto la capa de calles de OpenStreetMap, pero en la esquina superior derecha encontramos un control de capas en el que podemos seleccionar otras capas base:



Nota. Si nuestras capas tienen un SRID distinto al 4326, el mapa mostrará las capas, pero sin fondo.

El visor también incluye la herramienta “identificar”, que mostrará la información al hacer clic en un *feature* y veremos los datos de las columnas no geométricas:



Las consultas espaciales y funciones básicas

PostGIS añade cientos de funciones geométricas específicas a PostgreSQL.

Hablar de todas ellas (son más de 1000) está fuera del alcance de este curso, sin embargo, en esta sección vamos a trabajar en un pequeño ejemplo para introducirnos en la sintaxis de las funciones, y agregar otra tabla a nuestra base de datos *world*.

Cómo acceder a las funciones espaciales

El acceso a las funciones espaciales PostGIS no es diferente a acceder a las funciones de tipo texto, número o fecha en una base de datos común.

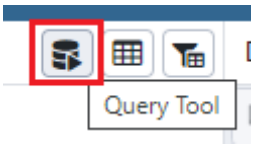
Las funciones se expresan en lenguaje SQL (*Structured Query Language*). SQL es un medio de hacer consultas o actualizar datos en bases de datos relacionales. Por ejemplo:

```
SELECT postgis_full_version();
```

Creación de una tabla habilitada para operaciones espaciales

Un pequeño ejemplo nos muestra cómo dotar de capacidades espaciales a una tabla recién creada en nuestra base de datos PostGIS utilizando la función **AddGeometryColumn**.

1. Si no lo está ya, abre tu ventana de consultas:



2. Copia y pega el siguiente bloque en el Editor de consultas SQL.

```
CREATE TABLE "lugares"
 (gid serial PRIMARY KEY,
 "nombre" varchar(50),
 "comentario" varchar(255),
 "año" numeric,
 geom geometry (Point, 4326)
);
```

3. Haz clic en el botón *Ejecutar consulta*  para ejecutar los comandos.

La primera línea en el código **CREATE TABLE...**, crea una tabla con las columnas especificadas y las claves. Estas son las acciones SQL sin componente espacial.

Fíjate en la última línea. Se crea el campo de geometría:

- Se especifica el tipo de geometría "Point",
- Y el identificador de referencia espacial "4326".

Si echamos un vistazo a los registros de la vista `geometry_colums`, podemos ver la fila de la tabla `lugares` habilitada para operaciones espaciales.

Ejercicio nº 6. Haz una captura de pantalla en la que se vea el nuevo registro y pégala en la plantilla de resultados.

A continuación, vamos a añadir algunas características a nuestra tabla `lugares`. Para añadir datos mediante SQL, utilizamos los comandos:

```
"INSERT INTO table (columns) VALUES (value)".
```

- *Table* es el nombre de la tabla en la que queremos añadir datos;
- *Columns* son los nombres de las columnas que queremos añadir a la tabla, pueden ser todas las columnas o solo parte de ellas;
- *Values* son los valores que queremos introducir en la tabla siguiendo el orden de las columnas.

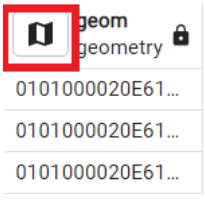
1. Si no lo está ya, abre tu ventana de consultas SQL pgAdmin.
2. Copia y pega el siguiente bloque en el Editor de consultas SQL.

```
INSERT INTO lugares (geom, nombre, comentario) VALUES
(ST_GeomFromText('POINT(-3.683352 40.400026)',4326), 'Madrid', 'Capital de Espana');

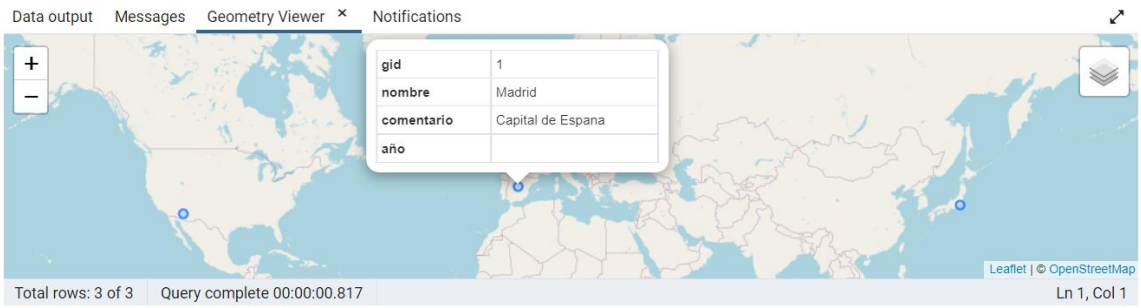
INSERT INTO lugares (geom, nombre, comentario) VALUES
(ST_GeomFromText('POINT(-112.069992 33.53998)',4326), 'Phoenix', 'Ciudad americana');

INSERT INTO lugares (geom, nombre, comentario) VALUES
(ST_GeomFromText('POINT (139.751407 35.685017)',4326), 'Tokyo', 'Capital de Japon');
```

3. Haz clic en el botón *Execute* para ejecutar los comandos.
4. Echa un vistazo de nuevo a la tabla `lugares`. Botón derecho en Tablas > Refresh...
5. Botón derecho en la tabla `lugares` > View/Edit Data> All Rows.
6. Desplázate hasta la pestaña geom y abre el visor de geometrías.



7. Haz clic sobre un elemento:



Ejercicio nº 7. Haz una captura de pantalla en la que se vean los nuevos registros y su información y pégala en la plantilla de resultados.

Una vez añadidos los objetos geográficos borra las sentencias anteriores del editor SQL.

Vamos a eliminar una de las filas. La sintaxis para borrar una fila es:

```
DELETE FROM lugares
WHERE gid= 3
```

Ejercicio nº 8. Ejecuta la consulta y haz una captura de pantalla con la fila eliminada. Pega la captura en la plantilla de resultados.

A continuación, vuelve a añadirla con el comando **INSERT INTO** que hemos visto anteriormente y los mismos datos.

Para cambiar el valor de una o varias columnas en una fila podemos utilizar el comando:

```
"UPDATE table SET column = value WHERE id_column = id".
```

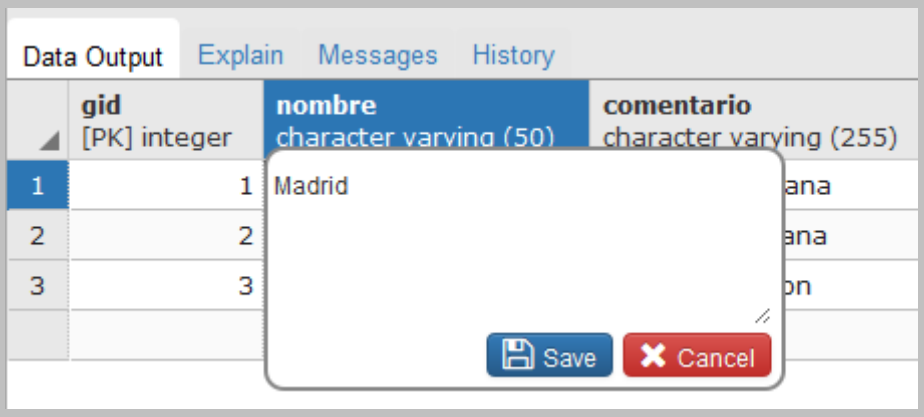
Utilizamos el valor de ID para identificar la fila, ya que es el único valor que no se puede repetir, es un identificador unívoco. Vamos a cambiar el valor del comentario de la ciudad de Phoenix. Para ello abrimos la ventana de consultas SQL y escribimos:

```
UPDATE  lugares
SET  comentario = 'capital del estado de Arizona'
WHERE gid = 2
```

Ejercicio nº 9. Ejecuta la consulta y pega el resultado en la plantilla de resultados.

Nota

Hemos visto como crear, actualizar y borrar filas en SQL, pero también puedes crear una nueva fila en la tabla, actualizar los valores o borrar la fila pinchando sobre la fila e interactuando con la tabla.

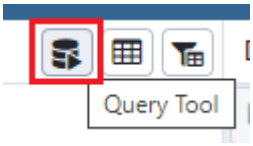


Consultas sencillas SQL

Ahora que hemos cargado datos en nuestra base de datos, vamos a utilizar SQL para hacer preguntas a los datos. Por ejemplo,

¿Cuáles son los nombres de todos los países?

Abre la ventana de consultas SQL en pgAdmin:



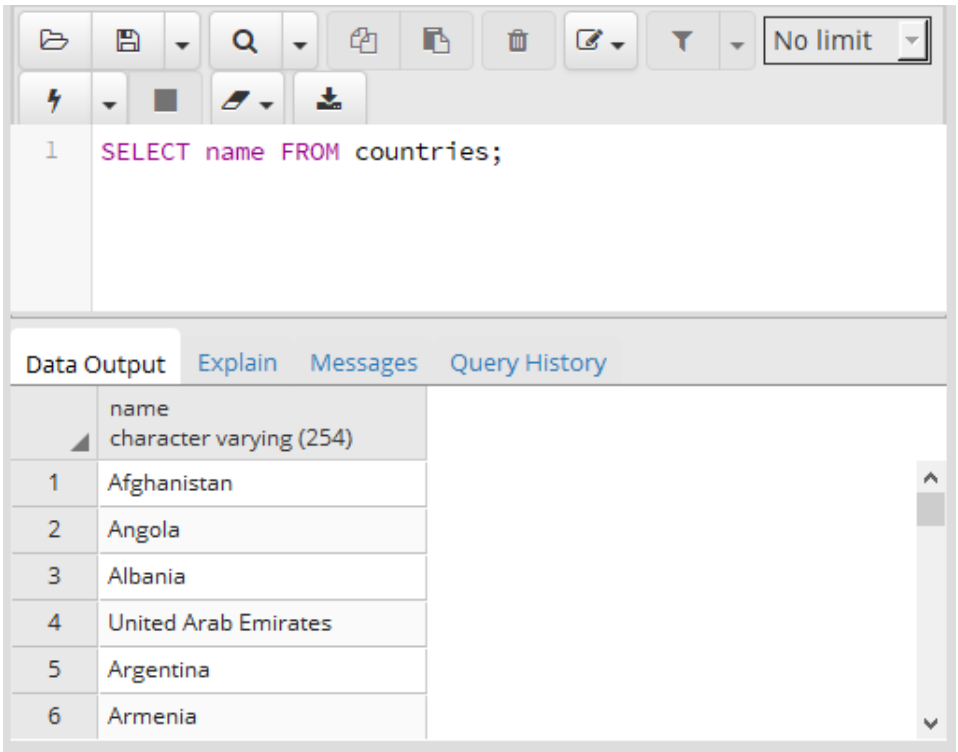
Después introduce la siguiente consulta en la ventana de consultas:

```
SELECT name FROM countries;
```

y pincha el botón *Ejecutar*



La consulta se ejecutará durante unos (mili) segundos y devolverá 177 resultados.



Cómo seleccionar datos de una tabla

Una consulta de selección generalmente tiene esta forma:

SELECT columna1, columna2 FROM nombretabla WHERE condición;

> **Nota**
>
> Para ver un resumen de todos los parámetros SELECT, consulta la documentación de PostgreSQL.

- Columna1, column2 son nombres de campos o funciones de valores de columna.
- nombretabla es o bien una sola tabla, o una tabla compuesta creada por la unión de dos tablas mediante una clave o condición.
- condición es un filtro que restringe el número de filas que se devuelven.

¿Cuáles son los nombres de los países en disputa?

Volvemos a nuestra tabla *countries*. La tabla contiene todos los países del mundo, pero solo queremos aquellos que están en disputa. Por lo tanto, deberemos crear un filtro:

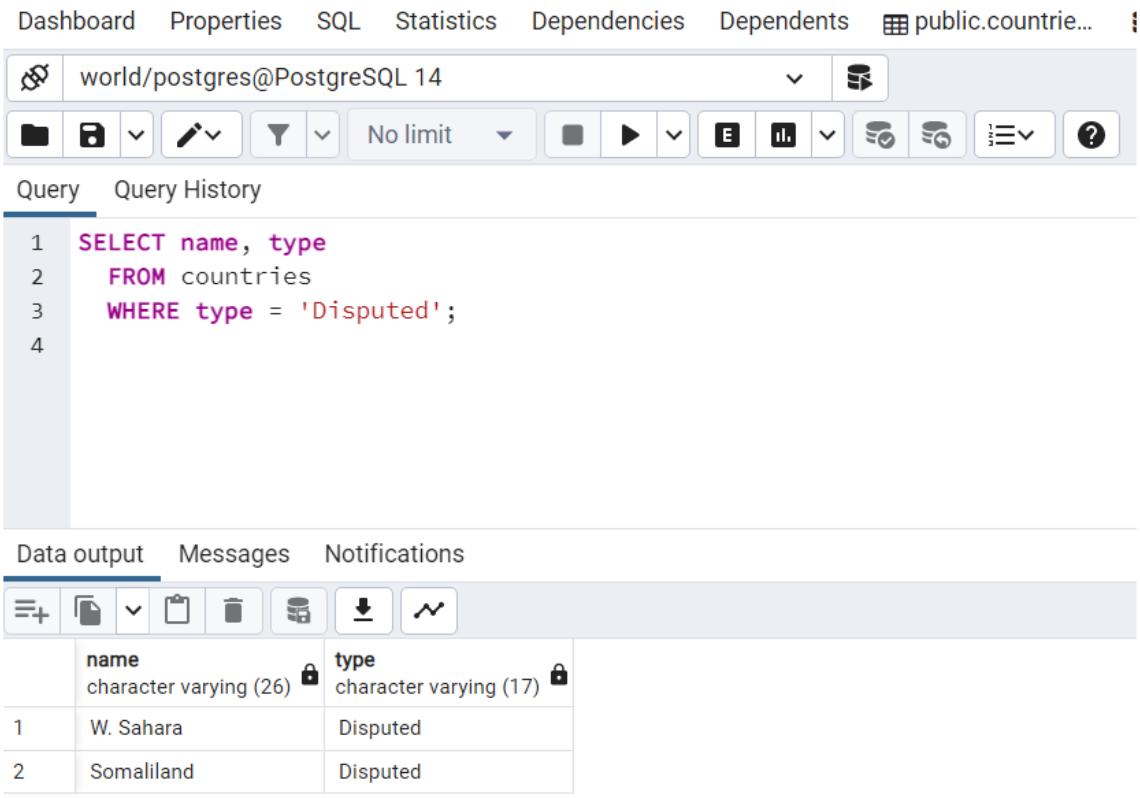
```
SELECT name
  FROM countries
  WHERE type = 'Disputed';
```

Seleccionamos la columna **name** de la tabla **countries** dónde la columna **type** sea igual a **Disputed**

La consulta devolverá 1 resultado.

En la sentencia SELECT podemos añadir varios campos para asegurarnos de que el resultado sea correcto:

```
SELECT name, type
  FROM countries
  WHERE type = 'Disputed';
```



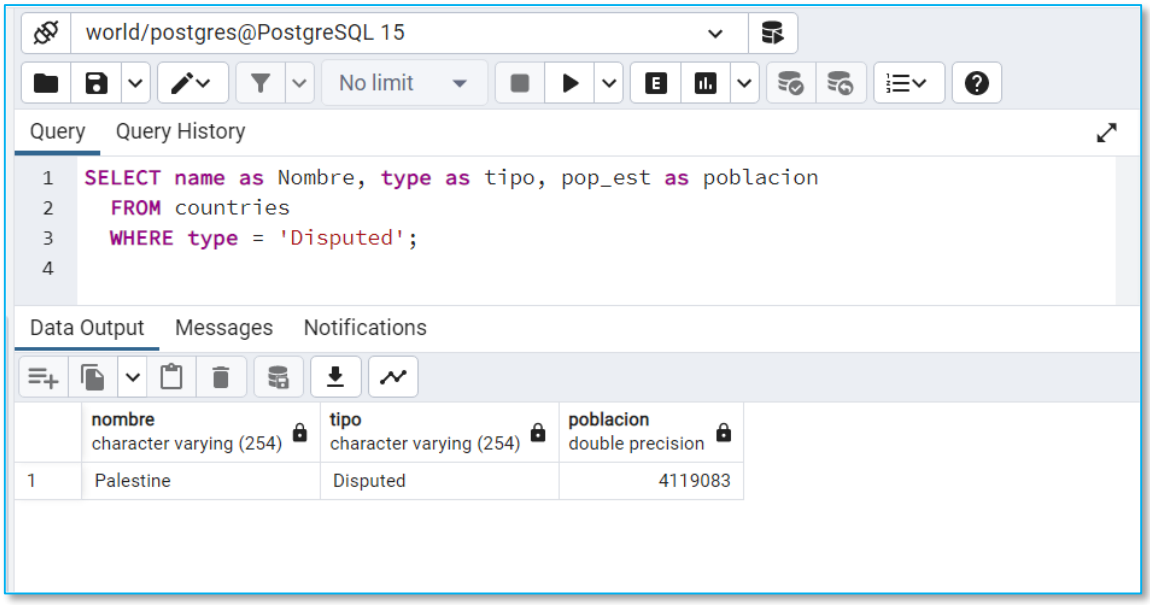
Uso de alias

Puedes dar a una tabla o columna otro nombre mediante el uso de un alias.

Los alias pueden hacer que las consultas sean más fáciles tanto de escribir como de leer.

Así que en lugar de obtener el nombre de nuestra columna, p.ej *pop* escribimos AS para que el nombre **población** sea más legible.

```
SELECT name as Nombre, type as tipo, pop_est as poblacion
 FROM countries
 WHERE type = 'Disputed';
```



A veces necesitaremos aplicar una función a los resultados de nuestra consulta. Por ejemplo,

¿Cuál es el número de letras de los nombres de los países en disputa?

Afortunadamente, PostgreSQL tiene una función de longitud de la cadena, **char_length (string)**.

```
SELECT char_length (name)
  FROM countries
  WHERE type = 'Disputed';
```

En otras ocasiones podemos estar más interesados en conocer las estadísticas que se aplican a varias filas y no en conocer solamente filas individuales.

Por lo tanto, conocer la longitud del nombre de los países puede ser menos interesante que conocer la longitud media de sus nombres. Las Funciones que utilizan varias filas y devuelven un único resultado son llamadas **funciones de "agregación**".

PostgreSQL tiene una serie de funciones de agregación integradas, incluyendo el uso general **avg ()** para valores medios, **stddev ()** para desviaciones estándar y **sum ()** para sumar un conjunto de registros.

¿Cuál es el número medio de letras y la desviación estándar del número de letras en el nombre de todos los países calificados como países?

```
SELECT avg(char_length(name)), stddev(char_length(name))
```

```
FROM countries
WHERE type = 'Country';
```

Data Output | Explain | Messages | History

	avg numeric	stddev numeric
1	9.2000000000000000	3.0110906108363240

Las funciones de agregación de nuestro último ejemplo se aplican a todas las filas del conjunto de resultados.

¿Qué pasa si queremos que los resúmenes se lleven a cabo a través de grupos más pequeños dentro del resultado global?

Para eso le debemos añadir una cláusula **GROUP BY**. Las funciones de agregación a menudo necesitan un añadir una sentencia **GROUP BY** para agrupar el conjunto de resultados de una o más columnas.

¿Cuál es el número medio de letras de los nombres de todos los países del mundo, reportado por tipo?

```
SELECT type, avg(char_length(name)), stddev(char_length(name))
 FROM countries
 GROUP BY type;
```

Incluimos la columna ***type*** en el resultado para que podamos determinar qué estadística se aplica a cada tipo. En una consulta de agregación, las columnas de salida sólo pueden ser o bien (a) miembros de la cláusula de la agrupación o (b) funciones de agregación.

Data Output | Explain | Messages | History

	type character varying (254)	avg numeric	stddev numeric
1	Sovereign country	7.7924528301886792	2.9662136963444120
2	Indeterminate	9.6666666666666667	0.57735026918962576451
3	Country	9.2000000000000000	3.0110906108363240
4	Dependency	14.5000000000000000	5.0662280511902212
5	Disputed	9.0000000000000000	[null]

Para realizar los ejercicios de Consultas vamos a utilizar la tabla `countries` que importamos anteriormente.

En primer lugar, vamos a seleccionar la población del país con mayor población. Para ello tenemos que escribir un código SQL. Utilizamos el comando *max*, para indicar el máximo valor, y la aplicamos a la columna *pop_est* (población) sobre la tabla `countries:`

```
SELECT max (pop_est) FROM countries
```

Ejercicio nº 10. Realiza una consulta de tal modo que devuelva una única fila con **el nombre** y población del país con la población más alta y pega el resultado en la plantilla de resultados.

Pista:

```
Nombre y la población
De la capa de países,
Dónde la población sea igual
      (a la máxima población de
      la capa de países)
```

Para seleccionar los países con más de 200 millones de habitantes, tenemos que seleccionar la columna *name* (nombre), e indicar que seleccione los registros en los que el campo *pop_est* sea mayor a 200.000.000.

Ejercicio nº 11. Haz una captura de pantalla con el resultado y pégala en la plantilla de resultados.

Para crear una lista ordenada utiliza el comando ORDER BY (ordenar por).

```
SELECT name
FROM countries
ORDER BY name
```

Funciones espaciales

Vamos a echar un vistazo a los diferentes tipos de funciones espaciales disponibles en PostGIS.

Funciones de conversión

Las funciones de conversión permiten que los datos se conviertan entre geometrías y formatos de datos externos

El siguiente ejemplo usa la función **ST_AsText** para descifrar la representación binaria.

1. Seleccionamos la geometría en binario de nuestra tabla lugares:

```
SELECT geom FROM lugares;
```

Obtenemos como resultado:

```
0101000020E6100000E8AAF9FF0D7662C0EA1099E022355040
0101000020E6100000CFC2DFBFD7D965402330B95D63A044C0
0101000020E6100000D753F09FD2365AC043259A5810D24340
```

2. Utiliza la función ST_AsText () para hacer que el valor de la geometría sea un poco más legible. Esta función nos devuelve el texto WKT (*Well-Known Text*):

```
SELECT ST_AsText(geom) FROM lugares;
```

Obtenemos como resultado:

```
"POINT(-3.683352 40.400026)"
"POINT(139.751407 35.685017)"
"POINT(-112.069992 33.53998)"
```

Ejercicio nº 12. Extrae las coordenadas geométricas de Madrid y pega el resultado obtenido en el Panel de salida en la plantilla de resultados.

Funciones de recuperación

Las funciones de recuperación nos muestran las medidas o propiedades de una geometría.

Utiliza la función **ST_Perimeter** para determinar la longitud exterior de algunos elementos de la tabla `countries`. Limita el resultado a 5 países.

Ejercicio nº 13. Crea la consulta y ejecútala. Pega el resultado obtenido en el Panel de salida en la plantilla de resultados.

Funciones de comparación

Las funciones de comparación evalúan relaciones espaciales entre dos geometrías.

Utiliza la función **ST_Distance** para calcular la distancia que hay entre Phoenix y Madrid.

La sintaxis será similar a esta (las coordenadas mostradas aquí no son correctas):

```
SELECT ST_Distance(
  ST_GeomFromText('POINT(14.8 39.64)'),
  ST_GeomFromText('POINT(73.91 40.17)')
);
```

`ST_GeometryFromText` devuelve el resultado en la unidad de la capa, (grados) `ST_GeographyFromText` devuelve el resultado en metros.

Ejercicio nº 14. Pega el resultado obtenido de la distancia expresada en metros entre Phoenix y Madrid. Estas ciudades están en la capa lugares.

Funciones de generación

Las funciones de generación crean geometrías a partir de otras.

Utilizaremos la función **ST_Centroid** para crear los puntos geométricos correspondientes a la capa de países. Llamaremos a la capa nueva `centroides.`

Para crear los centroides, primero crearemos una tabla de tipo punto para almacenar las geometrías.

```
CREATE TABLE centroides (
 id serial primary key,
geom geometry (Point, 4326)
);
```

Después, insertaremos en nuestra tabla las nuevas geometrías generadas con la función **ST_Centroid**:

```
INSERT INTO centroides (geom)
SELECT ST_Centroid(geom) FROM countries;
```

Si al ejecutar esta segunda sentencia aparece el error:

ERROR: Geometry SRID (0) does not match column SRID (4326)

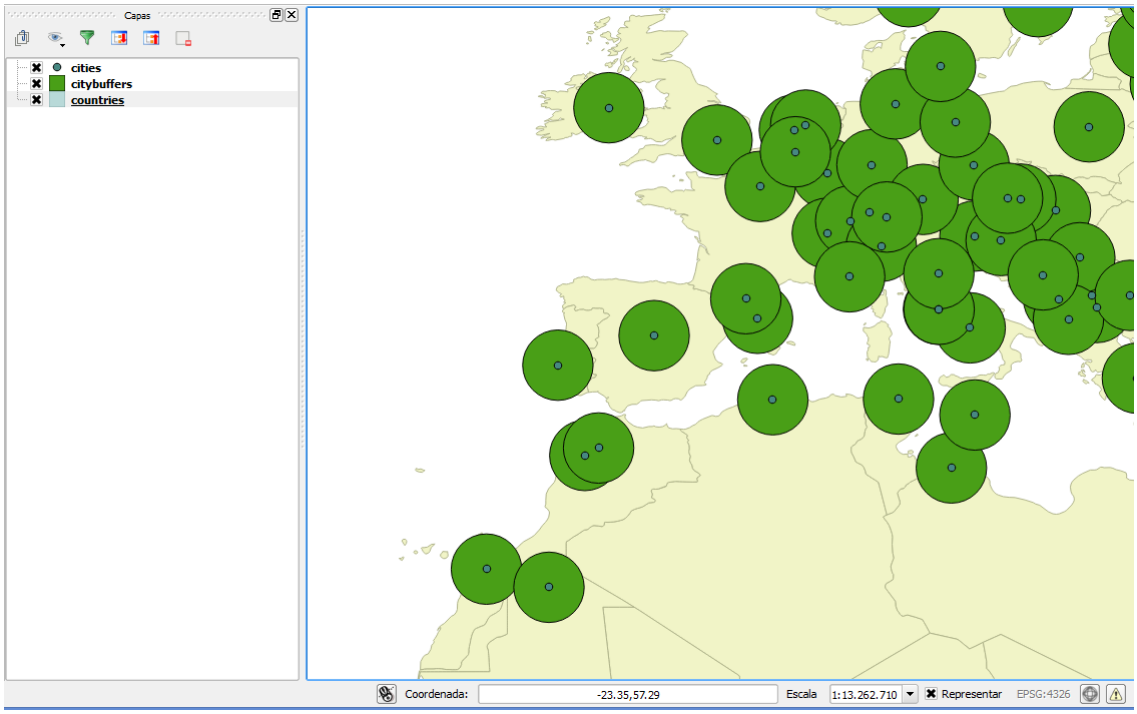
Probablemente cuando se realizó la importación de la tabla countries no se definió correctamente su SRID y se ha importado con el valor 0. Por lo tanto no es posible insertar en la capa de centroides (con SRID 4326) una geometría con valor de SRID diferente (0). Para solucionarlo tenemos dos opciones:

- Opción 1. Eliminar la tabla de la base de datos y volver a importarla introduciendo el código EPSG 4326.
- Opción 2. Actualizar el SRID mediante una sentencia SQL en el constructor de consultas de pgAdmin:

```
SELECT UpdateGeometrySRID('countries','geom',4326);
```

Ejercicio nº 15. Abre la nueva capa con un visor SIG, haz una captura de pantalla en la que se vean las capas de países y centroides.

Ahora debes crear un área de influencia (buffer) de 2 grados alrededor de nuestra capa *cities*.



Al igual que en el ejemplo anterior, para crear la zona de buffer, primero debemos crear una tabla para almacenar las geometrías. Denomínala buffer.

¿De qué tipo debe ser esta tabla?

A continuación, inserta en la nueva tabla las nuevas geometrías generadas.

¿Qué función debes utilizar?

Ejercicio nº 16. Añade a la plantilla de resultados los pasos que has dado y comprueba el resultado en un cliente SIG, mostrando la capa *cities* y la del buffer.

¿En qué país se encuentra la ciudad de Sao Paulo?

Para responder a esta pregunta debes realizar una intersección utilizando la función **ST_Intersects** (geometry A, geometry B) que devuelve verdadero si la geometría A intersecta con la geometría B.

Deberás **SELECT** el nombre del país **FROM** la tabla de países **WHERE** ST_Intersects (geometría de la tabla *countries*, coordenadas de Sao Paulo).

Ejercicio nº 17. Haz una captura de pantalla con la respuesta y las consultas utilizadas para resolver la pregunta.